

## Codebook of UltraGrammer

## Contents

## 1 解題流程

|                                   |   |
|-----------------------------------|---|
| 1.1 本 codebook 的用法                | 1 |
| 1.2 看 $N$ 決定複雜度 (1 秒約 $10^8$ 次運算) | 1 |
| 1.3 題目關鍵字 → 先想這些                  | 2 |
| 1.4 模板不能直接套時 (題目變形)               | 2 |
| 1.5 嘗試...                         | 2 |
| 1.6 檢查...                         | 2 |

## 2 基礎

|                                     |   |
|-------------------------------------|---|
| 2.1 模板                              | 2 |
| 2.2 Fast IO 與 <code>__int128</code> | 2 |
| 2.3 C++ 函式庫速查                       | 2 |
| 2.4 位元與數學小技巧                        | 2 |

## 3 資料結構

|                                   |   |
|-----------------------------------|---|
| 3.1 BIT (Fenwick Tree)            | 3 |
| 3.2 Segment Tree 線段樹 (lazy)       | 3 |
| 3.3 Sparse Table 稀疏表              | 3 |
| 3.4 Prefix Sum 前綴和與 Difference 差分 | 3 |
| 3.5 Monotonic Stack 單調棧           | 3 |
| 3.6 Monotonic Queue 單調隊列          | 4 |
| 3.7 Disjoint Set 並查集              | 4 |
| 3.8 座標壓縮、pb_ds 平衡樹                | 4 |

## 4 數學

|                              |   |
|------------------------------|---|
| 4.1 快速冪                      | 4 |
| 4.2 模逆元 (費馬小定理)              | 4 |
| 4.3 擴展歐幾里得                   | 4 |
| 4.4 組合數 $C(n, k)$            | 4 |
| 4.5 質數、質因數分解、線性篩             | 4 |
| 4.6 尤拉函數 $\varphi$           | 5 |
| 4.7 Legendre's Formula 勒讓德公式 | 5 |
| 4.8 數論分塊                     | 5 |
| 4.9 排容原理                     | 5 |
| 4.10 矩陣快速冪                   | 5 |
| 4.11 高斯消去、XOR 線性基            | 5 |
| 4.12 大數運算                    | 6 |
| 4.13 博弈論 (Nim、SG 函數)         | 6 |
| 4.14 常用公式                    | 6 |

## 5 字串

|                 |   |
|-----------------|---|
| 5.1 KMP 字串匹配    | 6 |
| 5.2 Z-Algorithm | 6 |
| 5.3 Manacher 迴文 | 6 |
| 5.4 Trie 字典樹    | 7 |
| 5.5 AC 自動機      | 7 |

## 6 圖論

|                             |   |
|-----------------------------|---|
| 6.1 BFS、拓撲排序、網格 BFS、0-1 BFS | 7 |
| 6.2 DFS                     | 7 |
| 6.3 Dijkstra 單源最短路          | 8 |
| 6.4 分層圖 / 狀態圖 (最短路 + 額外條件)  | 8 |
| 6.5 SPFA (Bellman-Ford)     | 8 |
| 6.6 Floyd-Warshall 全點對最短路   | 8 |
| 6.7 二分圖判定 (著色)              | 8 |
| 6.8 割點、橋、點雙連通分量             | 9 |
| 6.9 匈牙利演算法 (二分圖最大匹配)        | 9 |
| 6.10 KM 演算法                 | 9 |
| 6.11 Dinic 最大流              | 9 |

## 7 樹

|                     |    |
|---------------------|----|
| 7.1 Diameter 直徑     | 10 |
| 7.2 Traversal 遍歷    | 10 |
| 7.3 LCA 最近共同祖先 (倍增) | 10 |
| 7.4 Kruskal 最小生成樹   | 10 |
| 7.5 嚴格次小生成樹         | 10 |

## 8 幾何

|                         |    |
|-------------------------|----|
| 8.1 點與向量基礎              | 11 |
| 8.2 線段相交、直線交點           | 11 |
| 8.3 點與直線/線段             | 11 |
| 8.4 多邊形面積、點在多邊形內        | 11 |
| 8.5 凸包 Convex Hull、旋轉卡尺 | 11 |
| 8.6 圓                   | 12 |

## 9 DP

|                      |    |
|----------------------|----|
| 9.1 背包問題             | 12 |
| 9.2 分錢幣問題            | 12 |
| 9.3 LCS 最長共同子序列、編輯距離 | 12 |
| 9.4 LIS 最長遞增子序列      | 13 |
| 9.5 LCS 轉 LIS        | 13 |
| 9.6 區間 DP            | 13 |
| 9.7 加權區間排程 (工作有獎勵)   | 13 |
| 9.8 Kadane 最大子陣列和    | 14 |
| 9.9 跳格子 (路徑還原)       | 14 |

## 10 Greedy

|                      |    |
|----------------------|----|
| 10.1 區間覆蓋 (灑水器)      | 14 |
| 10.2 區間選點            | 14 |
| 10.3 最多不重疊區間         | 14 |
| 10.4 最少會議室           | 14 |
| 10.5 掃描線：最大重疊數       | 14 |
| 10.6 最小化最大延遲         | 14 |
| 10.7 有期限的單位工作排程      | 15 |
| 10.8 多機排程 (LPT 近似)   | 15 |
| 10.9 跳躍遊戲 (最少跳幾次)    | 15 |
| 10.10 Josephus 約瑟夫問題 | 15 |
| 10.11 Huffman 合併與編碼  | 15 |

## 11 搜尋與技巧

|                          |    |
|--------------------------|----|
| 11.1 Binary Search 二分搜   | 15 |
| 11.2 Ternary Search 三分搜  | 16 |
| 11.3 Two Pointer 雙指針     | 16 |
| 11.4 Sliding Window 滑動視窗 | 16 |
| 11.5 暴力枚舉、回溯、折半枚舉        | 16 |

## 12 MISC

|                  |    |
|------------------|----|
| 12.1 安全的 Hash    | 16 |
| 12.2 對拍器 (測資產生器) | 16 |

## 1 解題流程

## 1.1 本 codebook 的用法

- 每份 code 都假設已經有「模板」的 `bits/stdc++.h`、`using namespace std`、`typedef long long ll`。全部是**全域陣列** + **函式**，沒有 `struct`。
- 註解 `// [改]` 的那一行 = 題目變形時「通常要改的地方」。先找 `[改]`，再看檔案最後的 `[變形]` 清單。
- 每份檔案各自獨立。同時貼兩份時，重複的 `MAXN` / `INF` / `n` / `g` 要刪掉或改名。需要別的檔案時註解會寫「需要 XXX.cpp」。
- 多筆測資：全域陣列、`g[i].clear()`、計數器，每筆都要重設。

1.2 看  $N$  決定複雜度 (1 秒約  $10^8$  次運算)

| $N$ 的範圍             | 複雜度           | 可能的作法                  |
|---------------------|---------------|------------------------|
| $\leq 10$           | $O(N!)$       | 全排列、回溯                 |
| $\leq 20$           | $O(2^N N)$    | 位元枚舉                   |
| $\leq 40$           | $O(2^{N/2})$  | 折半枚舉                   |
| $\leq 500$          | $O(N^3)$      | Floyd、區間 DP、KM         |
| $\leq 5000$         | $O(N^2)$      | 二維 DP、LCS、枚舉兩端點        |
| $\leq 2 \cdot 10^5$ | $O(N \log N)$ | 排序、二分、BIT/線段樹、Dijkstra |
| $\leq 10^6$         | $O(N)$        | 雙指針、前綴和、KMP/Z、線性篩      |
| $\leq 10^{12}$      | $O(\sqrt{N})$ | 因數分解、數論分塊              |
| $\leq 10^{18}$      | $O(\log N)$   | 快速冪、矩陣快速冪、二分答案         |

### 1.3 題目關鍵字 → 先想這些

| 看到...                         | 想到...                     |
|-------------------------------|---------------------------|
| 最大值最小、最小值最大、至少需要多少            | 對答案二分搜 + greedy 檢查        |
| 最少步數 (每步代價相同)                 | BFS (網格、多源、0-1 BFS)       |
| 最短路 (有權 / 負權 / 全點對)           | Dijkstra / SPFA / Floyd   |
| 最短路 + 多一個條件 (次數、鑰匙、時間、油量)     | 分層圖 / 狀態圖                 |
| 先後順序、依賴關係、有沒有環                | 拓撲排序、DFS 判環               |
| 合併、是否同一組、連通塊                  | DSU、DFS                   |
| 花最少成本把所有點連起來                  | MST (Kruskal)             |
| 區間和、大量區間修改                    | 前綴和、差分、BIT、線段樹            |
| 區間最大最小 (不修改)、視窗最大             | Sparse Table、單調隊列         |
| 左/右邊第一個更大 (更小)                | 單調棧                       |
| 子陣列和 = $k$                    | 非負：雙指針；有負：前綴和 + map       |
| 找字串 / 多個字串 / 迴文               | KMP、Z / AC 自動機 / Manacher |
| 方法數 mod $p$ 、 $C(n, k)$ 、除法取模 | 組合數、模逆元                   |
| 遞迴式但 $n$ 到 $10^{18}$          | 矩陣快速幂                     |
| 選或不選、容量限制                     | 背包 DP                     |
| 配對、分派 (有權重 / 有容量)             | 匈牙利 (KM / Dinic)          |
| 拔掉後會斷開的點/邊                    | 割點、橋 (BCC)                |
| 樹上兩點路徑、距離                     | LCA (倍增)                  |
| 兩人輪流、誰必勝                      | Nim、SG 函數、打表找規律           |
| 排程、選最多不重疊區間                   | Greedy (依結束時間排序)          |
| 不重疊的工作、每個有獎勵，求最大獎勵            | 加權區間排程 DP                 |

### 1.4 模板不能直接套時 (題目變形)

- 多了一個條件 → 把它塞進「狀態」：最短路變成 (點, 條件) 的分層圖；DP 多開一維。狀態數  $\times$  轉移數  $\leq 10^8$  就能做。
- 終點固定、起點很多 → 反向建圖從終點跑一次；起點很多 → 超級源點或多源 BFS。
- 直接求很難、但「猜答案  $x$ 」後很好檢查 → 二分搜答案。
- 換個問法：總數 - 不合法數；最小割 = 最大流；最小點覆蓋 = 最大匹配；最大獨立集 =  $N$  - 匹配。
- 換枚舉的東西：枚舉答案、枚舉中間點、枚舉比較小的那一維；「所有子陣列的總和」→ 算每個元素被算幾次 (貢獻法)。
- 離線處理：把詢問排序；刪除很難 → 倒過來變加入 (DSU)。
- 化簡：先排序 (很多東西會變單調)； $|x_1 - x_2| + |y_1 - y_2|$  轉成  $(x + y, x - y)$  取 max；值域大但個數少 → 座標壓縮。
- 完全沒想法：寫暴力把小範圍答案印出來找規律 (打表)，再用對拍驗證。

### 1.5 嘗試...

- 再讀一遍題目，用範例手動模擬一次
- 懷疑你認為「理所當然」的直覺？試著寫下直覺的規律 (有助於觀察)
- 換個方向思考？暴力解？貪心解？對答案二分搜？動態規劃？
- 狀態建圖，把問題轉化成圖論？
- 只在奇數 index 反轉狀態？用乘法取代除法？( $/2 = \times 2^{-1}$ ，模逆元)

### 1.6 檢查...

- 確定沒漏看敘述？題目額外的敘述、暗示？輸出格式 (空白、換行、大小寫)？
- 數值範圍：溢位 (int\*int 先轉 1LL\*)、陣列大小、精度位數
- 多筆測資之間的初始化？ $N = 0$ 、 $N = 1$ 、全部相同、找不到答案時的輸出？
- 圖：不連通？自環、重邊？鏈狀樹 (遞迴太深)？ $V = 1, E = 0$ ？
- 模板抄錯？不同陣列/字串用同一個 index 而越界？非 void 函式忘記 return？
- 浮點數誤差 (用 eps，必要時 long double)？負數取模  $(x \% m + m) \% m$ ？除以 0？
- signed 與 unsigned 比較 (v.size()-1 在 size 為 0 時)？
- 區間的端點：開區間還是閉區間？同一時間點的事件誰先處理？

## 2 基礎

### 2.1 模板

用法：競賽用基礎模板。所有其他 code 都假設有這份的前 6 行。

```
1 #include <bits/stdc++.h>
2 #pragma GCC optimize("O3,unroll-loops")
3 // #pragma GCC target("avx2") // 評測機 CPU 支援 avx2，不確定就不要開
4 using namespace std;
5 typedef long long ll;
6 typedef pair<int, int> pii;
7 // 各模板自己會宣告 INF / MOD / MAXN; int 的 INF 用 0x3f3f3f3f (可 memset、相
  加不溢位)
8
9 int main() {
10     ios::sync_with_stdio(false); cin.tie(nullptr); // 之後不能混用
11     scanf/printf
12     int T = 1;
13     // cin >> T;
14     while (T--) {
15         // 每筆測資：全域陣列/vector/map 要清空！
16     }
17     // 讀到 EOF: while (cin >> n) { ... }
18     // 整行讀取: getline(cin, s); 前面有 cin >> 時先 cin.ignore()
19     // 浮點輸出: cout << fixed << setprecision(6) << x;
20     return 0;
21 }
```

### 2.2 Fast IO 與 \_\_int128

用法：輸入超過  $10^6$  個數時用 read(); \_\_int128 可存到約  $1.7 \times 10^{38}$ ，但要自己印。

```
1 // 輸入量超大 (> 1e6 個數) 才需要；一般 ios::sync_with_stdio(false) 就夠
2 inline ll read() {
3     ll x = 0; int c = getchar(); bool neg = false;
4     while (c != '-' && (c < '0' || c > '9')) c = getchar();
5     if (c == '-') neg = true, c = getchar();
6     while (c >= '0' && c <= '9') x = x * 10 + (c - '0'), c = getchar();
7     return neg ? -x : x;
8 }
9 // __int128 (約 ±1.7e38) 不能直接 cin/cout
10 void print128(__int128 x) {
11     if (x < 0) putchar('-'), x = -x;
12     if (x > 9) print128(x / 10);
13     putchar('0' + x % 10);
14 }
15 __int128 read128(const string &s) {
16     __int128 x = 0; int i = (s[0] == '-');
17     for (; i < (int)s.size(); i++) x = x * 10 + (s[i] - '0');
18     return s[0] == '-' ? -x : x;
19 }
```

### 2.3 C++ 函式庫速查

用法：常用 STL 與注意事項。

```
1 // <algorithm>
2 sort(a, a + n); sort(v.begin(), v.end(), greater<int>()); // 自訂: [] (auto
  &x, auto &y) { return ...; }
3 stable_sort(...); // 相等元素保持原
  順序
4 reverse(b, e); unique(b, e); // unique 前要先 sort，回傳新結尾:
  v.erase(unique(...), v.end())
5 lower_bound(b, e, x); // 第一個 >= x; upper_bound: 第一個 > x (都要已排序)
6 *min_element(b, e); *max_element(b, e); accumulate(b, e, 0LL); // 注意 0LL 才
  是 long long
7 next_permutation(b, e); // 下一個字典序排列，沒有了回傳 false
8 nth_element(b, b + k, e); // O(N) 把第 k 小放到位置 k
9 iota(b, e, 0); fill(b, e, x); count(b, e, x); find(b, e, x);
10 __gcd(a, b); gcd(a, b); lcm(a, b); // C++17 <numeric>
11 // <string>
12 s.substr(pos, len); s.find(t) (找不到 == string::npos); stoi(s); stoll(s);
  to_string(x);
13 // stringstream ss(line); while (ss >> word) 一行內數量不固定時
14 // 容器
15 // set/map: insert, erase(x), count, find, lower_bound(x), upper_bound(x)，全
  部 O(log N)
16 // multiset 只刪一個: ms.erase(ms.find(x)); ms.erase(x) 會刪掉全部的 x
17 // map 走訪: for (auto &[k, v] : mp); prev(s.end()) = 最大元素; *s.begin() =
  最小
18 // priority_queue: 預設 max-heap; min-heap: priority_queue<T, vector<T>,
  greater<T>>
19 // deque: push_front/back, pop_front/back, 也能 [i]
20 // bitset<N>: b.count(), b.set(i), b[i], b << k, b | c 一布林 DP 加速 64 倍
21 // vector<vector<int>> v(n, vector<int>(m, 0)); v.assign(n, x); v.resize(n);
22 // tuple: auto [a, b, c] = t; tie(a, b) = p;
```

### 2.4 位元與數學小技巧

```
1 /* 位元技巧
2 x & -x: 最低位的 1; x & (x-1): 去掉最低位的 1; __builtin_popcountll(x): 1
  的個數
3 __builtin_ctzll(x): 結尾 0 的個數; 63 - __builtin_clzll(x) (= -lg(x)): 最
  高位的位置
4 a ~ b ~ b = a; a + b = (a ^ b) + 2 * (a & b)
5 數學小技巧
6 ceil(a / b) (a, b > 0) = (a + b - 1) / b; 負數取模: ((a % m) + m) % m
7 排序比較 a/b < c/d < a*d < c*b (b, d > 0, 注意溢位)
8 long double 精度約 18 位; sqrt 整數: ll r = sqrtl(x); while (r*r > x) r--;
  while ((r+1)*(r+1) <= x) r++;
9 隨機: mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
  shuffle(v.begin(), v.end(), rng);
10 */
```

## 3 資料結構

### 3.1 BIT (Fenwick Tree)

**用法:** 單點修改、前綴和。1-based。變形：區間加區間和 (兩棵 BIT)、第  $k$  小、逆序對。

**時間複雜度:**  $O(\log N)$

```
1 // 1-based! n = 上界。每筆測資: fill(bit, bit+n+1, 0)
2 const int MAXN = 200005;
3 int n;
4 ll bit[MAXN];
5 void add(ll *t, int x, ll v) { for (; x <= n; x += x & -x) t[x] += v; }
6 ll sum(ll *t, int x) { ll r = 0; for (; x > 0; x -= x & -x) r += t[x]; return r; }
7 // 基本用法: 單點加 add(bit,i,v); 區間和 sum(bit,r) - sum(bit,l-1)
8
9 // [變形 1] 區間加 + 單點查: 對差分建 BIT
10 //   add(bit,l,v); add(bit,r+1,-v); a[x] = sum(bit,x)
11 // [變形 2] 區間加 + 區間和: 兩棵 BIT
12 ll B1[MAXN], B2[MAXN];
13 void range_add(int l, int r, ll v) {
14     add(B1, l, v); add(B1, r + 1, -v);
15     add(B2, l, v * (l - 1)); add(B2, r + 1, -v * r);
16 }
17 ll pre(int x) { return sum(B1, x) * x - sum(B2, x); }
18 ll range_sum(int l, int r) { return pre(r) - pre(l - 1); }
19
20 // [變形 3] 第 k 小: bit 存「值 x 出現幾次」, 回傳最小 x 使 sum(bit,x) >= k
21 int kth(ll k) {
22     int pos = 0;
23     for (int pw = 1 << _lg(n); pw; pw >= 1)
24         if (pos + pw <= n && bit[pos + pw] < k) pos += pw, k -= bit[pos];
25     return pos + 1; // 回傳 n+1 代表總數 < k
26 }
27 // [變形 4] 逆序對: 值先壓縮到 1..n, i=1..n 由左往右:
28 ans += (i-1) - sum(bit, a[i]); add(bit, a[i], 1); // 前面比 a[i] 大的個數
```

### 3.2 Segment Tree 線段樹 (lazy)

**用法:** 區間加值 + 區間和。改成區間 max/min 只要改標 [改] 的 4 行。1-based。

**時間複雜度:** 建樹  $O(N)$ , 修改/查詢  $O(\log N)$

```
1 // 1-based。區間加值 + 區間和 (lazy)。改成 max/min 只要改有 [改] 的行
2 const int MAXN = 200005;
3 int n;
4 ll a[MAXN], t[4 * MAXN], lz[4 * MAXN]; // t: 節點答案, lz: 還沒往下傳的加值
5
6 void apply_tag(int id, int l, int r, ll v) {
7     t[id] += v * (r - l + 1); // [改] max/min: t[id] += v;
8     lz[id] += v;
9 }
10 void pull(int id) { t[id] = t[id * 2] + t[id * 2 + 1]; } // [改]
11 void push(int id, int l, int r) {
12     if (lz[id] == 0) return;
13     int m = (l + r) / 2;
14     apply_tag(id * 2, l, m, lz[id]);
15     apply_tag(id * 2 + 1, m + 1, r, lz[id]);
16     lz[id] = 0;
17 }
18 void build(int id = 1, int l = 1, int r = n) {
19     lz[id] = 0;
20     if (l == r) { t[id] = a[l]; return; }
21     int m = (l + r) / 2;
22     build(id * 2, l, m); build(id * 2 + 1, m + 1, r);
23     pull(id);
24 }
25 void update(int ql, int qr, ll v, int id = 1, int l = 1, int r = n) { // [ql,qr] 加 v
26     if (qr < l || r < ql) return;
27     if (ql <= l && r <= qr) { apply_tag(id, l, r, v); return; }
28     push(id, l, r);
29     int m = (l + r) / 2;
30     update(ql, qr, v, id * 2, l, m); update(ql, qr, v, id * 2 + 1, m + 1, r);
31     pull(id);
32 }
33 ll query(int ql, int qr, int id = 1, int l = 1, int r = n) {
34     if (qr < l || r < ql) return 0; // [改] 單位元: sum 0, max -INF, min INF
35     if (ql <= l && r <= qr) return t[id];
36     push(id, l, r);
37     int m = (l + r) / 2;
38     return query(ql, qr, id * 2, l, m) + query(ql, qr, id * 2 + 1, m + 1, r);
39 } // [改]
40 // 用法: 讀 n, a[1..n] → build(); update(l,r,v); query(l,r)
41
42 /* [變形]
43 1. 區間「設值」: 另開 bool has[]。ll st[]
44    apply_tag: t[id] = v*(r-l+1) (max: v); st[id] = v; has[id] = 1; lz[id] = 0;
45    push: 先下傳 has/st, 再下傳 lz。同時有設值 + 加值時, 設值會清掉加值
46 2. 單點設值: update 走到葉子 (l==r) 直接 t[id] = v, 免 lazy
47 3. 需要合併多個資訊 (例: 最大子段和): 開 sum/pre/suf/best 四個陣列,
48    pull: sum=L.sum+R.sum, pre=max(L.pre,L.sum+R.pre),
49    suf=max(R.suf,R.sum+L.suf),
50    best=max({L.best,R.best,L.suf+R.pre})
51 4. 找第一個 >= x 的位置 (max 樹): 若 t[左] >= x 往左走, 否則往右走
52 5. 值域線段樹: 下標當「值」(先座標壓縮), t 存個數 → 可求第 k 小 / 區間內個數
53 */
```

### 3.3 Sparse Table 稀疏表

**用法:** 不修改的區間 max/min/gcd 查詢。0-based。

**時間複雜度:** 建表  $O(N \log N)$ , 查詢  $O(1)$

```
1 // 靜態 (不會修改) 區間 max/min/gcd 查詢。0-based
2 const int MAXN = 200005, LOG = 18; // 2^LOG > MAXN 即可 (1e6 用 20)
3 int st[LOG][MAXN];
4 void build(int n, int *a) {
5     for (int i = 0; i < n; i++) st[0][i] = a[i];
6     for (int j = 1; (1 << j) <= n; j++)
7         for (int i = 0; i + (1 << j) <= n; i++)
8             st[j][i] = max(st[j-1][i], st[j-1][i + (1 << (j-1))]); // [改] min/gcd
9 }
10 int query(int l, int r) { // [l, r]
11     int j = _lg(r - l + 1);
12     return max(st[j][l], st[j][r - (1 << j) + 1]); // [改]
13 }
14 // 注意: 只能用在「重疊不影響答案」的運算 (max/min/gcd/and/or), sum 不行
```

### 3.4 Prefix Sum 前綴和與 Difference 差分

**用法:** 前綴和: 多次區間查詢; 差分: 大量區間加值、最後才查。1D/2D 皆 1-based。

**時間複雜度:** 建表  $O(N)$ , 查詢  $O(1)$

```
1 // 全部 1-based, 讓 index 0 當作 0, 免判斷邊界
2 const int MAXN = 1005;
3 int n, m;
4 ll a[MAXN], pre[MAXN]; // 1D
5 ll A[MAXN][MAXN], P[MAXN][MAXN]; // 2D
6 ll d[MAXN], D[MAXN][MAXN]; // 差分
7
8 void build1() { for (int i = 1; i <= n; i++) pre[i] = pre[i-1] + a[i]; } // [改] XOR: ^
9 ll query1(int l, int r) { return pre[r] - pre[l-1]; }
10 // XOR: pre[r]^pre[l-1]
11
12 void build2() {
13     for (int i = 1; i <= n; i++)
14         for (int j = 1; j <= m; j++)
15             P[i][j] = P[i-1][j] + P[i][j-1] - P[i-1][j-1] + A[i][j];
16 }
17 ll query2(int x1, int y1, int x2, int y2) { // 左上 (x1,y1) 右下 (x2,y2)
18     return P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1];
19 }
20 // 1D 差分: 大量「區間加」, 最後才查
21 void add1(int l, int r, ll v) { d[l] += v; d[r+1] -= v; }
22 void restore1() { for (int i = 1; i <= n; i++) d[i] += d[i-1]; } // 之後 a[i] += d[i]
23
24 // 2D 差分
25 void add2(int x1, int y1, int x2, int y2, ll v) {
26     D[x1][y1] += v; D[x2+1][y1] -= v; D[x1][y2+1] -= v; D[x2+1][y2+1] += v;
27 }
28 void restore2() {
29     for (int i = 1; i <= n; i++)
30         for (int j = 1; j <= m; j++) D[i][j] += D[i-1][j] + D[i][j-1] - D[i-1][j-1];
31 }
32 /* [變形]
33 - 子陣列和 = k 的個數 (有負數): map<ll,int> cnt; cnt[0]=1; 每步 ans += cnt[pre-k]; cnt[pre]++;
34 - 子陣列和可被 k 整除: 同上, key 改成 ((pre%k)+k)%k
35 - 最長和為 0 的子陣列: 記錄每個 pre 值「第一次出現」的位置
36 */
```

### 3.5 Monotonic Stack 單調棧

**用法:** 左/右邊第一個比自己大 (小) 的位置。

**時間複雜度:**  $O(N)$

```
1 // 堆疊存 index。0(N)。0-based, 找不到為 -1
2 const int MAXN = 200005;
3 int n, a[MAXN], nxt[MAXN], prv[MAXN];
4 void next_greater() { // nxt[i] = i 右邊第一個「> a[i]」的位置
5     stack<int> st;
6     for (int i = 0; i < n; i++) {
7         while (!st.empty() && a[st.top()] < a[i]) { // [改] 下一個更小: >
8             nxt[st.top()] = i; st.pop();
9         }
10        st.push(i);
11    }
12    while (!st.empty()) nxt[st.top()] = -1, st.pop();
13 }
14 void prev_greater() { // prv[i] = i 左邊第一個「> a[i]」的位置
15     stack<int> st;
16     for (int i = 0; i < n; i++) {
17         while (!st.empty() && a[st.top()] <= a[i]) st.pop(); // [改] 前一個更
18        prv[i] = st.empty() ? -1 : st.top();
19        st.push(i);
20    }
21 }
22 /* [應用] 直方圖最大矩形: 對每個 i 找左右第一個「更矮」的 L,R,
23    面積 = a[i] * (R - L - 1)。以 a[i] 為最小值的最長區間」都是這招 */
```

### 3.6 Monotonic Queue 單調隊列

**用法:** 固定長度視窗的最大/最小值。

**時間複雜度:**  $O(N)$

```
1 // Sliding Window Maximum: 每個長度 k 的視窗的最大值。O(N)
2 const int MAXN = 200005;
3 int n, k, a[MAXN];
4 vector<int> window_max() {
5     deque<int> dq; // 存 index, 對應的值由前到後遞減
6     vector<int> res;
7     for (int i = 0; i < n; i++) {
8         while (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // 過期
9         while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); // [改] 最小值: >=
10        dq.push_back(i);
11        if (i >= k - 1) res.push_back(a[dq.front()]);
12    }
13    return res;
14 }
15 // [應用] DP 優化: dp[i] = max(dp[j]) + c, j in [i-k, i-1] → 對 dp 做 monotonic queue
```

### 3.7 Disjoint Set 並查集

**用法:** 合併集合、判斷是否同一組。路徑壓縮 + 小掛大。

**時間複雜度:** 均攤近似  $O(1)$

```
1 const int MAXN = 200005;
2 int f[MAXN], sz[MAXN];
3 void init(int n) { for (int i = 0; i <= n; i++) f[i] = i, sz[i] = 1; }
4 int find(int x) { return f[x] == x ? x : f[x] = find(f[x]); } // 路徑壓縮
5 bool unite(int a, int b) { // 回傳是否真的合併 (Kruskal 判環用)
6     a = find(a), b = find(b);
7     if (a == b) return false;
8     if (sz[a] < sz[b]) swap(a, b); // 小掛大
9     f[b] = a; sz[a] += sz[b];
10    return true;
11 }
12 /* [變形]
13 1. 集合大小: sz[find(x)]; 集合個數: 一開始 n, 每次 unite 成功就 -1
14 2. 帶權 DSU (x 到根的距離/差值 w[x]), find 時累加:
15    int find(int x){ if(f[x]==x) return x; int r=find(f[x]); w[x]+=w[f[x]];
16    return f[x]=r; }
17    (此時 find 不能先改 f[x]: 要先存 root 再加 w, 如上)
18 3. 二分圖/敵人關係: 開 2n 個點, x 與 y 敵對 → unite(x, y+n), unite(x+n, y)
19 4. 「下一個可用位置」(跳過已刪除): 刪 i 時 f[i] = i+1, find(i) 即右邊第一個沒刪的
20 5. 離線刪邊 → 反過來變成加邊
21 */
```

### 3.8 座標壓縮、pb\_ds 平衡樹

**用法:** 值很大但個數少 → 壓到  $1 \sim k$ 。ordered\_set 可查第  $k$  小與名次。

**時間複雜度:**  $O(N \log N)$

```
1 // 座標壓縮: 值很大 (1e9) 但個數少 → 映射到 1..k, 之後可開陣列/BIT
2 vector<ll> vals;
3 void compress(vector<ll> &a) {
4     vals = a;
5     sort(vals.begin(), vals.end());
6     vals.erase(unique(vals.begin(), vals.end()), vals.end());
7     for (ll &x : a) x = lower_bound(vals.begin(), vals.end(), x) -
8     vals.begin() + 1;
9 } // 還原: vals[id - 1]
10
11 // pb_ds 平衡樹 (可查名次的 set)
12 #include <ext/pb_ds/assoc_container.hpp>
13 #include <ext/pb_ds/tree_policy.hpp>
14 using namespace __gnu_pbds;
15 typedef tree<ll, null_type, less<ll>, rb_tree_tag,
16 tree_order_statistics_node_update> ordered_set;
17 // ordered_set os; os.insert(x); os.erase(x);
18 // *os.find_by_order(k): 第 k 小 (0-based); os.order_of_key(x): 有幾個 < x
19 // 要允許重複值: 改存 pair<ll, int>{值, 獨一無二的編號}
```

## 4 數學

### 4.1 快速冪

**用法:** 計算  $a^b \bmod m$ 。

**時間複雜度:**  $O(\log b)$

```
1 // (a^b) % mod, mod 常為 1e9+7 或 998244353
2 ll fast_pow(ll a, ll b, ll mod) {
3     ll res = 1; a %= mod;
4     if (a < 0) a += mod;
5     while (b > 0) {
6         if (b & 1) res = res * a % mod; // mod > 3e9 時改 (__int128)res * a % mod
7         a = a * a % mod;
8         b >>= 1;
9     }
10    return res;
11 }
12 // 費馬小定理降冪: p 質數、gcd(a,p)=1 時 a^b = a^(b mod (p-1)) (mod p)
13 // 歐拉降冪 (一般 m): b >= phi(m) 時 a^b = a^(b mod phi(m) + phi(m)) (mod m)
```

### 4.2 模逆元 (費馬小定理)

**用法:**  $p$  為質數且  $p \nmid a: a^{-1} \equiv a^{p-2} \pmod{p}$ , 所以  $x/a \equiv x \cdot a^{p-2}$ 。另附  $O(N)$  建逆元表。

**時間複雜度:**  $O(\log p)$

```
1 // 需要 fast_pow, p 必須是質數且 a % p != 0; 不是質數請用 exgcd 版
2 ll inv(ll a, ll p) { return fast_pow(a, p - 2, p); }
3 ll mod_div(ll a, ll b, ll p) { return (a % p + p) % p * inv(b, p) % p; } // (a/b) % p
4
5 // O(N) 建 1-n 的逆元表 (p 質數, n < p)
6 const int MAXN = 1000005;
7 ll invt[MAXN];
8 void build_inv(int n, ll p) {
9     invt[1] = 1;
10    for (int i = 2; i <= n; i++) invt[i] = (p - p / i) * invt[p % i] % p;
11 }
12 // 減法取模記得: ((a - b) % p + p) % p
```

### 4.3 擴展歐幾里得

**用法:** 解  $ax + by = \gcd(a, b)$ ; 模數不是質數時的模逆元。

**時間複雜度:**  $O(\log \min(a, b))$

```
1 // 求 ax + by = gcd(a,b) 的一組解, 回傳 gcd
2 ll extgcd(ll a, ll b, ll &x, ll &y) {
3     if (b == 0) { x = 1; y = 0; return a; }
4     ll g = extgcd(b, a % b, y, x);
5     y -= a / b * x;
6     return g;
7 }
8 /* 解 ax + by = c: g = gcd(a,b), c % g != 0 → 無解
9 否則 x0 = x*(c/g), y0 = y*(c/g); 通解 x = x0 + k*(b/g), y = y0 - k*(a/g)
10 最小非負 x: t = b/g; x = (x0 % t + t) % t (注意 x*(c/g) 可能溢位 →
11 __int128) */
12 // 模逆元 (m 不必是質數), 不存在回傳 -1
13 ll inverse(ll a, ll m) {
14     ll x, y;
15     if (extgcd(a, m, x, y) != 1) return -1;
16     return (x % m + m) % m;
17 }
```

### 4.4 組合數 $C(n, k)$

**用法:**  $n \leq 2000$  用帕斯卡 (模數可非質數);  $n \leq 10^6$  用階乘 + 逆元 (模數須為質數);  $n$  到  $10^{18}$ 、質數模數小用 Lucas。

**時間複雜度:**  $O(N^2) / O(N)$  建表、 $O(1)$  查詢

```
1 const ll MOD = 1e9 + 7; // [改] 依題目
2 // 一、帕斯卡三角形 (n <= 5000): 只有加法, MOD 不是質數也能用
3 const int MAXC = 2005;
4 int C[MAXC][MAXC];
5 void build_pascal() {
6     for (int i = 0; i < MAXC; i++) {
7         C[i][0] = 1;
8         for (int j = 1; j <= i; j++) C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % MOD;
9     }
10 }
11 // 二、階乘 + 逆元 (n <= 1e6), MOD 必須是質數。需要 fast_pow
12 const int MAXN = 1000005;
13 ll fact[MAXN], ifact[MAXN];
14 void build_fact() {
15     fact[0] = 1;
16     for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
17     ifact[MAXN - 1] = fast_pow(fact[MAXN - 1], MOD - 2, MOD);
18     for (int i = MAXN - 1; i > 0; i--) ifact[i - 1] = ifact[i] * i % MOD;
19 }
20 ll nCr(int n, int k) {
21     if (k < 0 || k > n) return 0;
22     return fact[n] * ifact[k] % MOD * ifact[n - k] % MOD;
23 }
24 // 三、Lucas: n, k 很大 (1e18) 但質數 p 小 (<= 1e6)。要先把 MOD 設成 p 再 build_fact
25 ll lucas(ll n, ll k, ll p) {
26     if (k == 0) return 1;
27     return nCr(n % p, k % p) * lucas(n / p, k / p, p) % p;
28 }
29 // 四、n 小 k 小但不取模: C(n, k) 用 res = res * (n-i) / (i+1) 逐步算 (先乘後除會整除)
```

### 4.5 質數、質因數分解、線性篩

**用法:** 單一數字  $O(\sqrt{N})$ ; 大量查詢用線性篩 (順便得到最小質因數 spf)。

**時間複雜度:**  $O(\sqrt{N}) /$  篩  $O(N)$

```
1 // 一、單一數字判斷 / 分解, 0 <= sqrt(N), N <= 1e12 可用 (更大用 Miller-Rabin)
2 bool is_prime(ll n) {
3     if (n < 2) return false;
4     for (ll i = 2; i * i <= n; i++) if (n % i == 0) return false;
5     return true;
6 }
7 vector<pair<ll, int>> factorize(ll n) { // {質因數, 次方}
8     vector<pair<ll, int>> res;
9     for (ll i = 2; i * i <= n; i++) {
10        if (n % i) continue;
11        int c = 0;
12        while (n % i == 0) n /= i, c++;
13        res.push_back({i, c});
14    }
15    if (n > 1) res.push_back({n, 1});
16    return res;
17 }
```

```

18 // 二、線性篩 O(N): 質數表 + 最小質因數 spf, N <= 1e7
19 const int MAXN = 10000005;
20 int spf[MAXN];
21 vector<int> primes;
22 void sieve(int n) {
23     for (int i = 2; i <= n; i++) {
24         if (spf[i] == 0) spf[i] = i, primes.push_back(i);
25         for (int p : primes) {
26             if (p > spf[i] || (1ll) i * p > n) break;
27             spf[i * p] = p;
28         }
29     }
30 }
31 // 有 spf 後，分解 x 只要 O(log x): while(x > 1){ int p = spf[x]; x /= p; ... }
32 // 因數個數 = Π(e_i + 1); 因數和 = Π(p^(e_i+1)-1)/(p-1)
33 // 區間篩 [L,R] (R 到 1e12, R-L 到 1e6): 用 sqrt(R) 內的質數去劃掉 [L,R] 內的倍數
34
35 /* 哥德巴赫: N 最少拆成幾個質數的和
36    N 是質數 + 1; N 偶數 (N>2) → 2; N 奇數且 N-2 是質數 + 2; 其他 + 3 */

```

| 範圍        | 擁有最多因數的數           | 因數個數   |
|-----------|--------------------|--------|
| $10^9$    | 735134400          | 1344   |
| $2^{31}$  | 2095133040         | 1600   |
| $10^{18}$ | 897612484786617600 | 103680 |

## 4.6 尤拉函數 $\varphi$

**用法:**  $1 \sim N$  中與  $N$  互質的個數; 另附  $1 \sim N$  全部建表。

**時間複雜度:**  $O(\sqrt{N})$  / 建表  $O(N \log \log N)$

```

1 // phi(n) = 1-n 中與 n 互質的個數, O(sqrt n)
2 ll phi(ll n) {
3     ll res = n;
4     for (ll i = 2; i * i <= n; i++) {
5         if (n % i) continue;
6         while (n % i == 0) n /= i;
7         res -= res / i; // res *= (1 - 1/i)
8     }
9     if (n > 1) res -= res / n;
10    return res;
11 }
12 // 1-n 全部的 phi, O(N log log N)
13 const int MAXN = 1000005;
14 int ph[MAXN];
15 void phi_table(int n) {
16     for (int i = 0; i <= n; i++) ph[i] = i;
17     for (int i = 2; i <= n; i++)
18         if (ph[i] == i) // i 是質數
19             for (int j = i; j <= n; j += i) ph[j] -= ph[j] / i;
20 }
21 // 性質: Σ_{d|n} phi(d) = n; 1-n 中與 n 互質的數總和 = n*phi(n)/2 (n>1)
22 // gcd(i,n)=d 的 i 個數 = phi(n/d) → Σ gcd(i,n) = Σ_{d|n} d*phi(n/d)

```

## 4.7 Legendre's Formula 勒讓德公式

**用法:**  $n!$  中質數  $p$  的次數  $v_p(n!) = \sum_{k \geq 1} \lfloor n/p^k \rfloor$ ;  $v_p\left(\frac{n!}{k!}\right) = v_p(n!) - v_p(k!)$ 。

**時間複雜度:**  $O(\log_p n)$

```

1 // n! 中質數 p 的次數 = Σ floor(n / p^k), O(log_p n)
2 ll legendre(ll n, ll p) {
3     ll ans = 0;
4     while (n) n /= p, ans += n;
5     return ans;
6 }
7 // C(n,k) 中 p 的次數
8 ll c_vp(ll n, ll k, ll p) { return legendre(n, p) - legendre(k, p) - legendre(n - k, p); }
9 /* 用途: n! 尾端 0 的個數 = legendre(n, 5)
10    m 是否整除 n!: 把 m 質因數分解, 每個 p^e 檢查 legendre(n,p) >= e
11    n! 在 b 進位的尾 0: b = Π p^e → min(legendre(n,p) / e) */

```

## 4.8 數論分塊

**用法:** 快速計算  $\sum_{i=1}^n \lfloor n/i \rfloor$  ( $\lfloor n/i \rfloor$  只有  $O(\sqrt{n})$  種值)。

**時間複雜度:**  $O(\sqrt{N})$

```

1 // Σ_{i=1..n} floor(n/i), O(sqrt N)。同一個區塊 [l, r] 的 n/i 都相同
2 ll math_block(ll n) {
3     ll ans = 0;
4     for (ll l = 1, r; l <= n; l = r + 1) {
5         r = n / (n / l); // 與 l 商相同的最右邊界
6         ans += (r - l + 1) * (n / l); // [改] Σ f(i)*floor(n/i): 改成
          (F(r)-F(l-1))*(n/l), F 為 f 的前綴和
7     }
8     return ans;
9 }
10 // 兩個: Σ floor(n/i)*floor(m/i) → r = min(n/(n/l), m/(m/l)), l 跑到 min(n,m)
11 // 只要 i 在 1..k (k<n): r = min(r, k)

```

## 4.9 排容原理

**用法:** 「至少滿足一個條件」的計數, 奇加偶減。

**時間複雜度:**  $O(2^M M)$

```

1 // 排容: 1-n 中能被 ps 中「至少一個」整除的個數。O(2^m * m)
2 ll inclusion_exclusion(ll n, const vector<ll> &ps) {
3     int m = ps.size();
4     ll ans = 0;
5     for (int mask = 1; mask < (1 << m); mask++) {
6         ll l = 1; int bits = 0;
7         for (int i = 0; i < m && l <= n; i++)
8             if (mask >> i & 1) {
9                 bits++;
10                l = (l > n / ps[i]) ? n + 1 : l * ps[i]; // [改] 不是質數要用 lcm; 防溢位
11            }
12        if (1 > n) continue;
13        ans += (bits & 1) ? n / l : -(n / l); // 奇加偶減
14    }
15    return ans;
16 }
17 /* 1-n 與 K 互質的個數 = n - inclusion_exclusion(n, K 的質因數)
18    錯排 D(n) = (n-1)(D(n-1)+D(n-2)), D(1)=0, D(2)=1
19    「每個條件都不滿足」= 總數 - 至少滿足一個; 「恰好 k 個」可用二項式反演 */

```

## 4.10 矩陣快速幂

**用法:** 線性遞迴  $f(n) = \sum c_i f(n-i)$  在  $n$  很大時。含取模。

**時間複雜度:**  $O(k^3 \log n)$

```

1 // 線性遞迴 f(n) = c1 f(n-1) + ... + ck f(n-k), n 到 1e18 → 矩陣快速幂 O(k^3 log n)
2 const ll MOD = 1e9 + 7;
3 typedef vector<vector<ll>> Mat;
4 Mat mul(const Mat &A, const Mat &B) { // (n x k) * (k x m)
5     int n = A.size(), k = B.size(), m = B[0].size();
6     Mat C(n, vector<ll>(m, 0));
7     for (int i = 0; i < n; i++)
8         for (int t = 0; t < k; t++) if (A[i][t])
9             for (int j = 0; j < m; j++) C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD;
10    return C;
11 }
12 Mat mpow(Mat A, ll p) { // A 必須是方陣
13     int n = A.size();
14     Mat R(n, vector<ll>(n, 0));
15     for (int i = 0; i < n; i++) R[i][i] = 1; // 單位矩陣
16     for (; p >= 1, A = mul(A, A)) if (p & 1) R = mul(R, A);
17     return R;
18 }
19 /* 例: Fibonacci [F(n+1)] [1 1]^n [F(1)]
20           [F(n)]   [1 0] [F(0)] → F(n) = mpow({{1,1},{1,0}}, n)[1][0]

```

一般  $k$  階: 第一列放係數  $c_1 \dots c_k$ , 下面是單位矩陣往下平移一格  
 有常數項 +d: 狀態向量多放一個 1, 矩陣多一行一列  
 圖上「恰好走  $k$  步」的路徑數 = 鄰接矩陣  $^k$ ; (min,+) 版本可求恰好  $k$  步最短路 \*/

## 4.11 高斯消去、XOR 線性基

**用法:** 解聯立方程組 (判斷無解/唯一/無限多解); 一堆數選子集的最大 XOR。

**時間複雜度:**  $O(N^3)$  /  $O(\log V)$  每次插入

```

1 // 高斯消去解 n 元一次方程組 A x = b (浮點數)。a[i][n] 放 b[i]
2 // 回傳: 0 無解, 1 唯一解 (存於 x), 2 無限多解
3 const double EPS = 1e-9;
4 int gauss(vector<vector<double>> a, vector<double> &x) {
5     int n = a.size(), m = a[0].size() - 1;
6     vector<int> where(m, -1);
7     for (int col = 0, row = 0; col < m && row < n; col++) {
8         int sel = row;
9         for (int i = row; i < n; i++) if (fabs(a[i][col]) > fabs(a[sel][col])) sel = i;
10        if (fabs(a[sel][col]) < EPS) continue;
11        swap(a[sel], a[row]);
12        where[col] = row;
13        for (int i = 0; i < n; i++) if (i != row) {
14            double c = a[i][col] / a[row][col];
15            for (int j = col; j <= m; j++) a[i][j] -= a[row][j] * c;
16        }
17        row++;
18    }
19    x.assign(m, 0);
20    for (int i = 0; i < m; i++) if (where[i] != -1) x[i] = a[where[i]][m] / a[where[i]][i];
21    for (int i = 0; i < n; i++) {
22        double s = 0;
23        for (int j = 0; j < m; j++) s += x[j] * a[i][j];
24        if (fabs(s - a[i][m]) > EPS) return 0;
25    }
26    for (int i = 0; i < m; i++) if (where[i] == -1) return 2;
27    return 1;
28 }
29 // 模質數版: 除法改成乘逆元、fabs>EPS 改成 != 0; XOR 版 (開關燈問題): 減法改 ^, 用 bitset
30
31 // XOR 線性基: 從一堆數選子集, XOR 最大值 / 能否湊出 x
32 ll basis[64];
33 bool insert_xor(ll x) {
34     for (int i = 63; i >= 0; i--) if (x >> i & 1) {
35         if (!basis[i]) { basis[i] = x; return true; }
36         x ^= basis[i];
37     }
38     return false; // x 已被湊出
39 }
40 ll max_xor() { ll r = 0; for (int i = 63; i >= 0; i--) r = max(r, r ^ basis[i]); return r; }

```

## 4.12 大數運算

**用法:** 加、減、比較、乘小數、乘大數、除以小數。先確認 `__int128` 夠不夠。

**時間複雜度:** 乘法  $O(NM)$ ，其餘  $O(N)$

```
1 // 大數: vector<int> 反向存 (個位在 [0])。先想想 __int128 (到 1.7e38) 夠不夠
2 typedef vector<int> Big;
3 Big to_big(const string &s) { Big a; for (int i = s.size() - 1; i >= 0; i--)
  a.push_back(s[i] - '0'); return a; }
4 void trim(Big &a) { while (a.size() > 1 && a.back() == 0) a.pop_back(); }
5 void print(const Big &a) { for (int i = a.size() - 1; i >= 0; i--) cout <<
  a[i]; cout << '\n'; }
6 int cmp(const Big &a, const Big &b) { // -1: a<b, 0: a==b, 1: a>b
  if (a.size() != b.size()) return a.size() < b.size() ? -1 : 1;
  for (int i = a.size() - 1; i >= 0; i--) if (a[i] != b[i]) return a[i] <
  b[i] ? -1 : 1;
  return 0; }
7 }
8 }
9 }
10 }
11 Big add(const Big &a, const Big &b) {
  Big c; int carry = 0;
  for (size_t i = 0; i < max(a.size(), b.size()) || carry; i++) {
  if (i < a.size()) carry += a[i];
  if (i < b.size()) carry += b[i];
  c.push_back(carry % 10); carry /= 10;
  }
  return c;
12 }
13 }
14 }
15 }
16 }
17 }
18 }
19 }
20 Big sub(const Big &a, const Big &b) { // 需 a >= b
  Big c = a; int borrow = 0;
  for (size_t i = 0; i < c.size(); i++) {
  c[i] -= borrow + (i < b.size() ? b[i] : 0);
  borrow = c[i] < 0;
  if (borrow) c[i] += 10;
  }
  trim(c); return c;
21 }
22 }
23 }
24 }
25 }
26 }
27 }
28 }
29 Big mul_small(const Big &a, ll b) { // b 到 1e9 都可
  Big c; ll carry = 0;
  for (size_t i = 0; i < a.size(); i++) {
  if (i < a.size()) carry += a[i] * b;
  c.push_back(carry % 10); carry /= 10;
  }
  trim(c); return c;
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 Big mul(const Big &a, const Big &b) { // O(N*M)
  vector<ll> c(a.size() + b.size(), 0);
  for (size_t i = 0; i < a.size(); i++)
  for (size_t j = 0; j < b.size(); j++) c[i + j] += a[i] * b[j];
  Big r; ll carry = 0;
  for (size_t i = 0; i < c.size(); i++) { carry += c[i]; r.push_back(carry
  % 10); carry /= 10; }
  trim(r); return r;
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 Big div_small(const Big &a, ll b, ll &rem) { // 商與餘數
  Big c(a.size()); rem = 0;
  for (int i = a.size() - 1; i >= 0; i--) { rem = rem * 10 + a[i]; c[i] =
  rem / b; rem %= b; }
  trim(c); return c;
46 }
47 }
48 }
49 }
```

## 4.13 博弈論 (Nim、SG 函數)

**時間複雜度:**  $O(N \cdot |\text{moves}|)$

```
1 /* 博弈 (兩人輪流、無法行動者輸)
2 - Nim: 多堆石頭任取  $a_1 \wedge a_2 \wedge \dots \wedge a_n \neq 0$  先手勝
3 - Bash: 一堆  $n$  個, 每次取  $1 \leq k \leq n$  ( $k+1$ )  $\neq 0$  先手勝
4 - 階梯 Nim: 只看奇數階的 XOR
5 - 不確定時: 小範圍暴力算 win[] 找規律 (打表!)
6 - 多個獨立遊戲的組合  $\rightarrow$  各自算 SG 值再 XOR,  $\neq 0$  先手勝 */
7 const int MAXN = 100005;
8 int sg[MAXN];
9 vector<int> moves = {1, 3, 4}; // [改] 每次可拿的數量
10 void build_sg(int n) {
  for (int i = 0; i <= n; i++) {
  set<int> s; // 所有後繼狀態的 SG 值
  for (int m : moves) if (i >= m) s.insert(sg[i - m]);
  int g = 0;
  while (s.count(g)) g++; // mex
  sg[i] = g;
  }
11 }
12 }
13 }
14 }
15 }
16 }
17 }
18 }
19 // sg[i] == 0  $\rightarrow$  必敗態 (P-position);  $\neq 0 \rightarrow$  必勝態
```

## 4.14 常用公式

- Catalan:  $C_n = \frac{1}{n+1} \binom{2n}{n}$ , 1, 1, 2, 5, 14, 42, 132。合法括號、二元樹個數、不超過對角線的路徑。
- 第二類 Stirling ( $n$  個相異球放進  $k$  個相同非空盒):  $S(n, k) = kS(n-1, k) + S(n-1, k-1)$ 。
- 錯排:  $D_n = (n-1)(D_{n-1} + D_{n-2})$ ,  $D_1 = 0, D_2 = 1$ 。
- 插板法:  $x_1 + \dots + x_k = n$  的非負整數解  $\binom{n+k-1}{k-1}$ ; 正整數解  $\binom{n-1}{k-1}$ 。
- 格子路徑  $(0, 0) \rightarrow (a, b)$  只往右/上:  $\binom{a+b}{a}$ 。
- $\sum i^2 = \frac{n(n+1)(2n+1)}{6}$ ,  $\sum i^3 = \left(\frac{n(n+1)}{2}\right)^2$ , 等比  $\sum_{i=0}^{n-1} r^i = \frac{r^n - 1}{r - 1}$ 。
- 費馬:  $a^{p-1} \equiv 1$ ; 尤拉:  $a^{\varphi(m)} \equiv 1 \pmod{m}$  ( $\gcd(a, m) = 1$ ); Wilson:  $(p-1)! \equiv -1 \pmod{p}$ 。

- $\gcd(a, b) = \gcd(b, a \bmod b)$ ,  $\text{lcm}(a, b) = a / \gcd(a, b) \cdot b$  (先除再乘防溢位)。
- 平面圖 Euler:  $V - E + F = 2$  (連通,  $F$  含外部面)。Pick:  $A = I + B/2 - 1$ 。

## 5 字串

### 5.1 KMP 字串匹配

**用法:**  $\text{pi}[i] = s[0..i]$  最長「相等真前後綴」的長度。回傳所有 0-based 起點; 也可求最小循環節。

**時間複雜度:**  $O(N + M)$

```
1 // pi[i] = s[0..i] 最長「相等真前後綴」的長度
2 vector<int> prefix_function(const string &s) {
  int n = s.size();
  vector<int> pi(n, 0);
  for (int i = 1; i < n; i++) {
  int j = pi[i - 1];
  while (j > 0 && s[i] != s[j]) j = pi[j - 1]; // 失配就往回跳
  if (s[i] == s[j]) j++;
  pi[i] = j;
  }
  return pi;
12 }
13 // p 在 t 中所有出現的起點 (0-based) 0<|t|+|p|
14 vector<int> kmp(const string &t, const string &p) {
  vector<int> pi = prefix_function(p), res;
  for (int i = 0, j = 0; i < (int)t.size(); i++) {
  while (j > 0 && t[i] != p[j]) j = pi[j - 1];
  if (t[i] == p[j]) j++;
  if (j == (int)p.size()) { res.push_back(i - j + 1); j = pi[j - 1]; }
  }
  return res;
21 }
22 }
23 // 最小循環節:  $k = n - \text{pi}[n-1] \circ n \% k == 0 \rightarrow s$  由  $s.\text{substr}(0, k)$  重複  $n/k$  次;
  否則答案是  $n$ 
24 int min_period(const string &s) {
  int n = s.size(), k = n - prefix_function(s).back();
  return n % k == 0 ? k : n;
25 }
26 }
27 }
28 /* [變形]
29 - 不重疊計數: 匹配成功後  $j = 0$  (而不是  $\text{pi}[j-1]$ )
30 -  $s$  的所有「既是前綴也是後綴」的長度:  $j = \text{pi}[n-1]$ , 一直  $j = \text{pi}[j-1]$  直到 0
31 - 最短補字元變迴文 (補在後面):  $t = \text{rev}(s) + \text{'\#'} + s$ , 答案  $n - \text{pi}.back()$ 
32 */
```

### 5.2 Z-Algorithm

**用法:**  $z[i] = s$  與  $s[i..]$  的最長共同前綴。0-based。

**時間複雜度:**  $O(N)$

```
1 // z[i] = s 與 s[i..] 的最長共同前綴長度 (z[0] 定義為 0)  $\circ O(N)$ 
2 vector<int> z_function(const string &s) {
  int n = s.size();
  vector<int> z(n, 0);
  for (int i = 1, l = 0, r = 0; i < n; i++) { // [l, r] 為目前最右的匹配區間
  if (i <= r) z[i] = min(r - i + 1, z[i - l]);
  while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
  if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
  }
  return z;
11 }
12 }
13 // 字串匹配:  $s = p + \text{'\$'} + t$  ('\$' 不能出現在 p, t 中)
14 vector<int> z_match(const string &t, const string &p) {
  vector<int> z = z_function(p + '$' + t), res;
  for (int i = p.size() + 1; i < (int)z.size(); i++)
  if (z[i] == (int)p.size()) res.push_back(i - p.size() - 1);
  return res;
17 }
18 }
19 // 最小週期: 最小的 i 使  $i + z[i] == n$  (且  $n \% i == 0$  才是整除循環)
```

### 5.3 Manacher 迴文

**用法:** 最長迴文子字串、迴文子字串總數、 $O(1)$  判斷  $s[l..r]$  是否迴文。

**時間複雜度:**  $O(N)$

```
1 // 把 "abc" 變成 "$#a#b#c#~", 奇偶長度統一處理。p[i] = 以 t[i] 為中心的半徑
2 // p[i] 剛好等於「原字串中對應迴文的長度」, 起點 = (i - p[i]) / 2
3 string t;
4 vector<int> p;
5 void manacher(const string &s) {
  t = "$#";
  for (char c : s) t += c, t += '#';
  t += '~';
  int n = t.size(), c = 0, r = 0;
  p.assign(n, 0);
  for (int i = 1; i < n - 1; i++) {
  if (i < r) p[i] = min(r - i, p[2 * c - i]); // 抄鏡像
  while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) p[i]++; // 暴力擴展
  if (i + p[i] > r) c = i, r = i + p[i];
  }
  }
16 }
17 string longest_palindrome(const string &s) {
  manacher(s);
  int best = 1;
  for (int i = 1; i + 1 < (int)t.size(); i++) if (p[i] > p[best]) best = i;
  return s.substr((best - p[best]) / 2, p[best]);
22 }
23 ll count_palindromes(const string &s) { // 迴文子字串總數 (位置不同算不同)
  manacher(s);
  ll total = 0;
  for (int i = 1; i + 1 < (int)t.size(); i++) total += (p[i] + 1) / 2;
24 }
25 }
26 }
```

```

27     return total;
28 }
29 // s[l..r] 是否迴文: 中心 i = l + r + 2, 檢查 p[i] >= r - l + 1

```

## 5.4 Trie 字典樹

**用法:** 字串出現次數、前綴個數、刪除; SIGMA=2 時可做最大 XOR。  
root = 0。

**時間複雜度:**  $O(|S|)$

```

1 // root = 0, 節點從 1 開始編號; tr[u][c] == 0 代表沒有這條路
2 const int MAXN = 1000005, SIGMA = 26; // MAXN = 所有字串總長 + 1; [改] 二進位
3 XOR: SIGMA = 2
4 int tr[MAXN][SIGMA], pass_cnt[MAXN], end_cnt[MAXN], cnt = 0;
5 void trie_init() { // 多筆測資: 只清用過的節點
6     for (int i = 0; i <= cnt; i++) memset(tr[i], 0, sizeof tr[i]),
7         pass_cnt[i] = end_cnt[i] = 0;
8     cnt = 0;
9 }
10 void insert(const string &s) {
11     int u = 0; pass_cnt[u]++;
12     for (char ch : s) {
13         int c = ch - 'a'; // [改] 字元集
14         if (!tr[u][c]) tr[u][c] = ++cnt;
15         u = tr[u][c]; pass_cnt[u]++;
16     }
17     end_cnt[u]++;
18 }
19 int walk(const string &s) { // 走到 s 的節點, 不存在回傳 -1
20     int u = 0;
21     for (char ch : s) { u = tr[u][ch - 'a']; if (!u) return -1; }
22     return u;
23 }
24 int count_word(const string &s) { int u = walk(s); return u == -1 ? 0 :
25     end_cnt[u]; } // s 出現幾次
26 int count_prefix(const string &s) { int u = walk(s); return u == -1 ? 0 :
27     pass_cnt[u]; } // 以 s 為前綴的個數
28 void erase(const string &s) { // 需確定 s 存在
29     int u = 0; pass_cnt[u]--;
30     for (char ch : s) { u = tr[u][ch - 'a']; pass_cnt[u]--; }
31     end_cnt[u]--;
32 }
33 // [變形] 最大 XOR 配對: SIGMA=2, 把數字從高位 (bit 30 → 0) 插入;
34 // 查 x 時每一位盡量走「相反」的 bit: int b = x >> i & 1; if (tr[u][!b] &&
35     pass_cnt[tr[u][!b]]) res |= 1 < i, u = tr[u][!b]; else u = tr[u][b];

```

## 5.5 AC 自動機

**用法:** 多個 pattern 同時匹配, 求每個 pattern 在文章中出現幾次。fail[v] 指向 v 的最長且存在於 Trie 中的真後綴。

**時間複雜度:**  $O(\text{總長} \cdot \Sigma + |T|)$

```

1 // 多個 pattern 同時在文章中匹配。root = 0
2 const int MAXN = 1000005, SIGMA = 26; // MAXN = pattern 總長 + 1
3 int tr[MAXN][SIGMA], fail[MAXN], occ[MAXN], cnt = 0;
4 vector<int> order_; // BFS 順序
5 int insert(const string &s) { // 回傳 s 的結尾節點 (記下來查答案用)
6     int u = 0;
7     for (char ch : s) {
8         int c = ch - 'a';
9         if (!tr[u][c]) tr[u][c] = ++cnt;
10        u = tr[u][c];
11    }
12    return u;
13 }
14 void build() { // 全部 insert 完再呼叫
15     queue<int> q;
16     for (int c = 0; c < SIGMA; c++) if (tr[0][c]) fail[tr[0][c]] = 0,
17         q.push(tr[0][c]);
18     while (!q.empty()) {
19         int u = q.front(); q.pop(); order_.push_back(u);
20         for (int c = 0; c < SIGMA; c++) {
21             int v = tr[u][c];
22             if (v) fail[v] = tr[fail[u]][c], q.push(v);
23             else tr[u][c] = tr[fail[u]][c]; // 不存在的路直接連到 fail 的對應點
24         }
25     }
26 } // 文章 text 跑一次。之後 pattern i 出現次數 = occ[end_node[i]] * 0(|text| + 總長)
27 void query(const string &text) {
28     int u = 0;
29     for (char ch : text) u = tr[u][ch - 'a'], occ[u]++;
30     for (int i = (int)order_.size() - 1; i >= 0; i--) { // 由深到淺, 把次數加到 fail
31         int v = order_[i];
32         occ[fail[v]] += occ[v];
33     }
34 }
35 /* [變形]
36 - 只問「有幾個 pattern 出現過」: 看 occ[end_node[i]] > 0 的個數
37 - 文章不能包含任何 pattern (禁止字串 DP): 先把 bad[v] |= bad[fail[v]] (按 BFS 順序),
38     dp[長度][AC 節點], 轉移 v = tr[u][c], 跳過 bad[v] 的狀態
39 */

```

## 6 圖論

表示圖: 鄰接串列 vector<int> g[MAXN] (帶權用 pair<int, ll>)、鄰接矩陣 (Floyd /  $V \leq 500$ )、邊列表 (Kruskal)。注意: 有向/無向、有沒有權重、是否連通、自環與重邊。連通平面圖的區域數可用  $V - E + F = 2$ 。

## 6.1 BFS、拓樸排序、網格 BFS、0-1 BFS

**用法:** 無權最短路; 拓樸排序 (可判環、DAG DP); 網格四方向; 邊權只有 0/1 時用 deque。

**時間複雜度:**  $O(V + E)$

```

1 const int MAXN = 200005;
2 int n, m, indeg[MAXN], dist[MAXN];
3 vector<int> g[MAXN]; // 多筆測資: for(i=0..n) g[i].clear(), indeg[i]=0;
4
5 // 1. 拓樸排序 (Kahn)。回傳順序, size < n 代表有環
6 vector<int> topo_sort() {
7     queue<int> q; vector<int> res; // [改] 要字典序最小:
8     priority_queue<int, vector<int>, greater<int>>
9     for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
10    while (!q.empty()) {
11        int u = q.front(); q.pop(); res.push_back(u);
12        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
13    }
14    return res;
15 }
16 // DAG 上 DP: 依拓樸順序轉移, 例如最長路 dp[v] = max(dp[v], dp[u] + w) 路徑數
17 dp[v] += dp[u]
18
19 // 2. BFS: 無權圖最短路 (邊權全為 1)
20 void bfs(int s) { // [改] 多源 BFS: 把所有起點一開始都
21     push, dist = 0
22     fill(dist, dist + n + 1, -1);
23     queue<int> q; q.push(s); dist[s] = 0;
24     while (!q.empty()) {
25         int u = q.front(); q.pop();
26         for (int v : g[u]) if (dist[v] == -1) dist[v] = dist[u] + 1,
27             q.push(v);
28     }
29 }
30
31 // 3. 網格 BFS
32 const int MAXR = 1005;
33 int R, C, gd[MAXR][MAXR];
34 char grid[MAXR][MAXR];
35 int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1}; // [改] 8 方向 / 騎士走法
36 void grid_bfs(int sx, int sy) {
37     for (int i = 0; i < R; i++) fill(gd[i], gd[i] + C, -1);
38     queue<pair<int, int>> q; q.push({sx, sy}); gd[sx][sy] = 0;
39     while (!q.empty()) {
40         auto [x, y] = q.front(); q.pop();
41         for (int d = 0; d < 4; d++) {
42             int nx = x + dx[d], ny = y + dy[d];
43             if (nx < 0 || nx >= R || ny < 0 || ny >= C) continue;
44             if (grid[nx][ny] == '#' || gd[nx][ny] != -1) continue; // [改] 障礙物條件
45             gd[nx][ny] = gd[x][y] + 1; q.push({nx, ny});
46         }
47     }
48 }
49
50 // 4. 0-1 BFS: 邊權只有 0 或 1, 用 deque, 權 0 推前面、權 1 推後面, O(V+E)
51 vector<pair<int, int>> wg[MAXN];
52 int d01[MAXN];
53 void bfs01(int s) {
54     fill(d01, d01 + n + 1, INT_MAX);
55     deque<int> dq; dq.push_back(s); d01[s] = 0;
56     while (!dq.empty()) {
57         int u = dq.front(); dq.pop_front();
58         for (auto [v, w] : wg[u]) if (d01[u] + w < d01[v]) {
59             d01[v] = d01[u] + w;
60             if (w) dq.push_back(v); else dq.push_front(v);
61         }
62     }
63 }
64
65 // 例: 網格「最少折幾道牆」/ 「最少轉向次數」→ 走空地權 0、折牆/轉向權 1

```

## 6.2 DFS

**用法:** 連通塊、有向圖判環 (三色)、小  $N$  枚舉所有路徑 (回溯)。

**時間複雜度:**  $O(V + E)$

```

1 const int MAXN = 200005;
2 int n, comp[MAXN], color[MAXN];
3 vector<int> g[MAXN];
4 // 1. 連通塊: comp[v] = 所屬編號 (0 = 未拜訪)
5 void dfs(int u, int id) {
6     comp[u] = id;
7     for (int v : g[u]) if (!comp[v]) dfs(v, id);
8 }
9
10 int count_components() {
11     fill(comp, comp + n + 1, 0);
12     int id = 0;
13     for (int i = 1; i <= n; i++) if (!comp[i]) dfs(i, ++id);
14     return id;
15 }
16
17 // 2. 有向圖找環: 0 未拜訪, 1 在遞迴堆疊中, 2 已完成。走到狀態 1 的點 = 有環
18 bool has_cycle(int u) {
19     color[u] = 1;
20     for (int v : g[u]) {
21         if (color[v] == 1) return true;
22         if (color[v] == 0 && has_cycle(v)) return true;
23     }
24     color[u] = 2;
25     return false;
26 }
27
28 // 無向圖找環: 走到「已拜訪且不是父節點」的點即有環; 或 DSU unite 失敗
29 // 3. 小  $N$  (<= 10) 枚舉所有路徑: 回溯
30 bool vis[MAXN];
31 ll best;
32 void all_paths(int u, ll len) {
33     best = max(best, len); // [改] 依題意更新答案 (最長/最短/計數)
34     for (int v : g[u]) if (!vis[v]) {
35

```

```

32     vis[v] = true; all_paths(v, len + 1);
33     vis[v] = false; // 回溯：一定要還原
34 }
35 }
36 // 遞迴太深 (鏈狀 1e5 以上) 可能 stack overflow → 改用 stack 的迭代版或 BFS

```

## 6.3 Dijkstra 單源最短路

**用法:** 邊權  $\geq 0$ 。附路徑還原、最短路條數、瓶頸路寫法與常見變形。

**時間複雜度:**  $O((V + E) \log V)$

```

1 // 單源最短路，邊權 >= 0。O((V+E) log V)
2 const int MAXN = 200005;
3 const ll INF = 1e18;
4 int n, par[MAXN];
5 ll dist[MAXN], ways[MAXN];
6 vector<pair<int, ll>> g[MAXN]; // g[u] = {v, w}; 無向圖兩邊都加
7 void dijkstra(int s) { // [改] 多源：所有起點 dist = 0 並 push
8     fill(dist, dist + n + 1, INF);
9     fill(ways, ways + n + 1, 0);
10    fill(par, par + n + 1, -1);
11    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
12    int>>> pq; // {距離, 點}
13    dist[s] = 0; ways[s] = 1; pq.push({0, s});
14    while (!pq.empty()) {
15        auto [d, u] = pq.top(); pq.pop();
16        if (d > dist[u]) continue; // 過期的資料
17        for (auto [v, w] : g[u]) {
18            ll nd = d + w; // [改] 瓶頸路 (路徑上最大邊最小): nd =
19            max(d, w)
20            if (nd < dist[v]) {
21                dist[v] = nd; par[v] = u; ways[v] = ways[u];
22                pq.push({nd, v});
23            } else if (nd == dist[v]) {
24                ways[v] = (ways[v] + ways[u]) % 1000000007; // 最短路條數 (邊
25                權需 > 0)
26            }
27        }
28    }
29    vector<int> get_path(int t) { // 先確認 dist[t] != INF
30        vector<int> path;
31        for (int v = t; v != -1; v = par[v]) path.push_back(v);
32        reverse(path.begin(), path.end());
33        return path;
34    }
35    /* [常見變形]
36    - 所有點到「同一個終點」：建反向圖，從終點跑一次
37    - 必經某點 k: dist(s,k) + dist(k,t)，從 s 和 k 各跑一次
38    - 某條邊 (u,v,w) 是否在某條最短路上: ds[u] + w + dt[v] == ds[t] (ds 從 s, dt
39    從 t 在反向圖)
40    - 點也有權重：把點權加到「進入該點的邊」上，或拆點 (v_in → v_out 權 = 點權)
41    - 多一個條件 (次數/時間/燃料/鑰匙...) → 看「分層圖 / 狀態圖」那一節
42    - 第 K 短路 (可重複點)：每個點最多被 pop K 次，第 K 次 pop 到終點即答案
43    */

```

## 6.4 分層圖 / 狀態圖 (最短路 + 額外條件)

**用法:** 題目多一個條件時，把「(點, 條件)」當成新的點。方法一直接建大圖再呼叫原本的 Dijkstra；方法二把條件放進 dist 的第二維。

**時間複雜度:**  $O(K(V + E) \log(KV))$

```

1 /* 分層圖 / 狀態圖：題目在最短路上「多一個條件」時用
2 核心：狀態 = (點, 額外資訊)。把每個狀態當成一個新的點，演算法本身不用改！
3 例 (NCPK 型)：最多 K 次可以讓一條邊免費 → 狀態 (v, 已用次數 k)
4 (u,k) → (v,k) 權 w : 正常走
5 (u,k) → (v,k+1) 權 0 : 用一次特權 (k < K)
6 答案 = min over k of dist(t, k)
7 */
8 // 方法一 (推薦)：直接建出大圖，然後呼叫原本的 Dijkstra (Dijkstra 的 MAXN 要開
9 >= N*(K+1)+1)
10 int N, K; // 原圖點數 (1-based)、特權次數
11 int id(int v, int k) { return k * N + v; }
12 void build_layered(const vector<array<ll, 3>> &edges) { // 原圖的邊 {u, v, w}
13     n = N * (K + 1); // Dijkstra 用的 n 變成「狀態總數」
14     for (int i = 0; i <= n; i++) g[i].clear();
15     for (auto [u, v, w] : edges)
16         for (int k = 0; k <= K; k++) {
17             g[id(u, k)].push_back({id(v, k), w}); // [改] 同層
18             // 轉移
19             if (k < K) g[id(u, k)].push_back({id(v, k + 1), 0}); // [改] 跨層
20             // 轉移 (折半: w/2)
21             // 無向圖：反方向 (v→u) 也要加同樣兩條
22         }
23 }
24 // 用法: build_layered(edges); dijkstra(id(s, 0));
25 // ll ans = INF; for (k = 0..K) ans = min(ans, dist[id(t, k)]); [改]
26 // 必須恰好用 K 次 → 只看 k = K
27 // 方法二：狀態放在 dist 的第二維 (狀態太多、或轉移是「當場算出來」時，例如網格 +
28 // 鑰匙)
29 const int MAXV = 100005, MAXK = 11;
30 ll ds[MAXV][MAXK];
31 vector<pair<int, ll>> adj[MAXV];
32 void state_dijkstra(int s) {
33     for (int i = 0; i <= N; i++) fill(ds[i], ds[i] + K + 1, INF);
34     priority_queue<tuple<ll, int, int>, vector<tuple<ll, int, int>>,
35     greater<tuple<ll, int, int>>> pq;
36     ds[s][0] = 0; pq.push({0, s, 0});
37     auto relax = [&](int v, int k, ll nd) { if (nd < ds[v][k]) ds[v][k] = nd,
38     pq.push({nd, v, k}); };
39     while (!pq.empty()) {
40         auto [d, u, k] = pq.top(); pq.pop();
41         if (d > ds[u][k]) continue;

```

```

36     for (auto [v, w] : adj[u]) {
37         relax(v, k, d + w);
38         if (k < K) relax(v, k + 1, d); // [改] 一般轉移
39     }
40 }
41 }
42 /* 常見的「額外資訊」(狀態數 × 轉移數 要 <= 約 1e7-1e8):
43 | 題目條件 | 狀態 |
44 | 最多 K 次免費/折扣/瞬移 | (v, 已用次數) |
45 | 收集鑰匙開門 (鑰匙 <= 10 種) | (x, y, 鑰匙 bitmask) |
46 | 油箱容量 C、加油站 | (v, 剩餘油量) |
47 | 紅綠燈/週期 T 的時間限制 | (v, 時間 mod T) |
48 | 走奇數/偶數步、交替顏色邊 | (v, 奇偶 / 上一條邊顏色) |
49 | 轉彎次數、不能連續同方向 | (x, y, 方向) |
50 | 是否經過特定點/是否已用技能 | (v, 0/1) |
51 邊權全 1 → BFS; 只有 0/1 → 0-1 BFS; 一般非負 → Dijkstra
52 狀態之間若是 DAG (例如 k 只會增加且同層無環) 也可以直接 DP
53 */

```

## 6.5 SPFA (Bellman-Ford)

**用法:** 可有負權邊、可判負環、差分約束。只有在有負邊時用。

**時間複雜度:** 平均快，最差  $O(VE)$

```

1 // 可有負權邊的單源最短路 + 負環偵測。平均快，最差 O(VE)
2 const int MAXN = 100005;
3 const ll INF = 1e18;
4 int n, cnt[MAXN];
5 ll dist[MAXN];
6 bool inq[MAXN];
7 vector<pair<int, ll>> g[MAXN];
8 bool spfa(int s) { // false = 從 s 走得到負環
9     fill(dist, dist + n + 1, INF);
10    fill(inq, inq + n + 1, false);
11    fill(cnt, cnt + n + 1, 0);
12    queue<int> q;
13    dist[s] = 0; q.push(s); inq[s] = true;
14    while (!q.empty()) {
15        int u = q.front(); q.pop(); inq[u] = false;
16        for (auto [v, w] : g[u]) if (dist[u] + w < dist[v]) {
17            dist[v] = dist[u] + w;
18            if (!inq[v]) {
19                if (++cnt[v] >= n) return false; // 入隊 >= n 次 → 負環
20                q.push(v); inq[v] = true;
21            }
22        }
23    }
24    return true;
25 }
26 /* [變形]
27 - 判斷「整張圖」有沒有負環：建超級源點 0 連到每個點權 0，從 0 跑 (n 要算進 0 →
28 用 n+1 判斷)
29 - 差分約束：條件 x[v] - x[u] <= w → 邊 u→v 權 w。超級源點 0 → 每點權 0
30 有負環 = 無解；否則 dist 就是一組解 (x[v]-x[u] >= w 改寫成 x[u]-x[v] <= -w)
31 - 最長路：權重取負後跑最短路 (正環 = 最長路無限大)
32 */

```

## 6.6 Floyd-Warshall 全點對最短路

**用法:** k 一定在最外層。附路徑還原、負環、遞移閉包。

**時間複雜度:**  $O(V^3)$

```

1 // 全點對最短路 O(V^3), V <= 500。可有負邊 (不可有負環)
2 const int MAXN = 505;
3 const ll INF = 1e18;
4 int n, nxt[MAXN][MAXN];
5 ll d[MAXN][MAXN];
6 void init() {
7     for (int i = 1; i <= n; i++)
8         for (int j = 1; j <= n; j++) d[i][j] = (i == j ? 0 : INF), nxt[i][j]
9         = j;
10 }
11 // 讀邊: d[u][v] = min(d[u][v], w); (重邊取 min! 無向圖兩個方向都設)
12 void floyd() {
13     for (int k = 1; k <= n; k++) // k (中繼點) 一定在最外層
14         for (int i = 1; i <= n; i++) if (d[i][k] < INF)
15             for (int j = 1; j <= n; j++) if (d[k][j] < INF && d[i][k] +
16             d[k][j] < d[i][j]) {
17                 d[i][j] = d[i][k] + d[k][j]; // [改] 瓶頸: max(d[i][k],
18                 d[k][j]) 再取 min
19                 nxt[i][j] = nxt[i][k]; // 路徑還原
20             }
21 }
22 vector<int> path(int i, int j) { // 需 d[i][j] < INF
23     vector<int> p = {i};
24     while (i != j) i = nxt[i][j], p.push_back(i);
25     return p;
26 }
27 /* [變形]
28 - 有負環：跑完後 d[i][i] < 0 的點在負環上
29 - 遞移閉包 (誰能到誰): bool 版 reach[i][j] |= reach[i][k] && reach[k][j], 可用
30 bitset 加速
31 - 最小路 (無向): 在 k 迴圈「更新前」先算 min(d[i][j] + w[i][k] + w[k][j]),
32 i < j < k
33 */

```

## 6.7 二分圖判定 (著色)

**用法:** 相鄰點塗不同色，衝突 = 有奇環。已處理不連通。

**時間複雜度:**  $O(V + E)$

```

1 // 二分圖判定 (相鄰點塗不同色)，已處理不連通。color: 0 未塗, 1 / -1
2 const int MAXN = 200005;
3 int n, color_[MAXN];
4 vector<int> g[MAXN];
5 bool is_bipartite() {
6     fill(color_, color_ + n + 1, 0);
7     for (int s = 1; s <= n; s++) {
8         if (color_[s]) continue;
9         queue<int> q; q.push(s); color_[s] = 1;
10        while (!q.empty()) {
11            int u = q.front(); q.pop();
12            for (int v : g[u]) {
13                if (!color_[v]) color_[v] = -color_[u], q.push(v);
14                else if (color_[v] == color_[u]) return false; // 奇環
15            }
16        }
17    }
18    return true;
19 }
20 // 二分圖 <=> 沒有奇數環。分兩組/兩隊/兩種顏色的題目先想這個
21 // 每個連通塊可以「翻轉顏色」→ 各塊 (黑數, 白數) 再做背包 DP 求最平均分組

```

## 6.8 割點、橋、點雙連通分量

**用法:** 一次 Tarjan 找出割點、橋與所有 v-BCC (孤立點自成一個)。用邊編號處理重邊。

**時間複雜度:**  $O(V + E)$

```

1 /* 無向圖。dfn[u]: DFS 拜訪順序; low[u]: u 的子樹「不經過父邊」能回到的最小 dfn
2 割點: 非根 u 有子節點 v 使 low[v] >= dfn[u]; 根需 >= 2 個 DFS 子樹
3 點雙連通分量 (v-BCC): 內部任兩點有兩條點不重複路徑; 割點可同時屬於多個 BCC */
4 const int MAXN = 200005;
5 int n, timer_, dfn[MAXN], low[MAXN];
6 bool is_cut[MAXN];
7 vector<pair<int, int>> g[MAXN]; // {鄰點, 邊編號}, 邊編號用來處理重邊
8 vector<vector<int>> bcc;
9 vector<int> stk;
10 void add_edge(int u, int v, int id) { g[u].push_back({v, id});
11   g[v].push_back({u, id}); }
12 void tarjan(int u, int pe) { // pe = 走進 u 的邊編號
13     dfn[u] = low[u] = ++timer_;
14     stk.push_back(u);
15     int child = 0;
16     for (auto [v, id] : g[u]) {
17         if (id == pe) continue;
18         if (dfn[v]) { low[u] = min(low[u], dfn[v]); continue; } // 回邊
19         child++;
20         tarjan(v, id);
21         low[u] = min(low[u], low[v]);
22         if (low[v] >= dfn[u]) {
23             if (pe != -1) is_cut[u] = true;
24             vector<int> comp = {u}; // u 留在 stack 上 (可能還屬於別的 BCC)
25             while (true) {
26                 int w = stk.back(); stk.pop_back();
27                 comp.push_back(w);
28                 if (w == v) break;
29             }
30             bcc.push_back(comp);
31         }
32     }
33     if (pe == -1 && child >= 2) is_cut[u] = true;
34 }
35 void solve() {
36     timer_ = 0; bcc.clear();
37     fill(dfn, dfn + n + 1, 0); fill(is_cut, is_cut + n + 1, false);
38     for (int i = 1; i <= n; i++) if (!dfn[i]) {
39         stk.clear();
40         tarjan(i, -1);
41         if (g[i].empty()) bcc.push_back({i}); // 孤立點自成一個 BCC
42     }

```

## 6.9 匈牙利演算法 (二分圖最大匹配)

**用法:** 左右兩側, match\_r[v] 記錄右邊 v 配到誰。附二分圖重要定理。  
**時間複雜度:**  $O(VE)$

```

1 // 二分圖最大匹配 (Kuhn),  $O(V * E)$ 。左邊 1..nl, 右邊 1..nr
2 const int MAXN = 1005;
3 int nl, nr, match_r[MAXN]; // match_r[v] = 右邊 v 配到的左邊點 (0 = 沒配)
4 bool vis[MAXN];
5 vector<int> g[MAXN]; // g[左 u] = 可配的右邊點
6 bool try_match(int u) {
7     for (int v : g[u]) {
8         if (vis[v]) continue;
9         vis[v] = true;
10        if (!match_r[v] || try_match(match_r[v])) { match_r[v] = u; return
11            true; } // 搶位子
12    }
13    return false;
14 }
15 int max_matching() {
16     fill(match_r, match_r + nr + 1, 0);
17     int res = 0;
18     for (int u = 1; u <= nl; u++) {
19         fill(vis, vis + nr + 1, false);
20         if (try_match(u)) res++;
21     }
22     return res;
23 }
24 /* 二分圖定理 (重要!):
25 最小點覆蓋 = 最大匹配 (König)
26 最大獨立集 = 總點數 - 最大匹配

```

```

26 DAG 最短路徑覆蓋 (路徑不相交) = 點數 - 最大匹配 (每點拆成 左 u、右 u, 邊 u→v
27 變 左 u→右 v)
28 點數很大 (1e6) 改用 Dinic, 複雜度  $O(E \sqrt{V})$  */

```

## 6.10 KM 演算法

**用法:** 帶權二分圖最大權完美匹配, 左右各  $N$  點 (不足補虛點)。

**時間複雜度:**  $O(N^3)$

```

1 // 帶權二分圖「最大權完美匹配」, 左右各 n 點,  $O(n^3)$ 。1-based
2 // 點數不同或沒有邊: 補虛擬點/權重 0 的邊 (不能選的邊設 -INF 級大負數)
3 // 要「最小權»: 權重全部取負, 答案再取負
4 const int MAXN = 505;
5 const ll INF = 1e18;
6 int n, match_y[MAXN], pre_[MAXN], slack[MAXN];
7 ll w[MAXN][MAXN], lx[MAXN], ly[MAXN], slack[MAXN];
8 bool vis_y[MAXN];
9 void km_bfs(int root) {
10    fill(slack, slack + n + 1, INF);
11    fill(vis_y, vis_y + n + 1, false);
12    fill(pre_, pre_ + n + 1, 0);
13    int y = 0;
14    match_y[0] = root;
15    while (true) {
16        int x = match_y[y], ny = 0;
17        ll delta = INF;
18        vis_y[y] = true;
19        for (int i = 1; i <= n; i++) if (!vis_y[i]) {
20            ll d = lx[x] + ly[i] - w[x][i];
21            if (d < slack[i]) slack[i] = d, pre_[i] = y;
22            if (slack[i] < delta) delta = slack[i], ny = i;
23        }
24        for (int i = 0; i <= n; i++) {
25            if (vis_y[i]) lx[match_y[i]] -= delta, ly[i] += delta;
26            else slack[i] -= delta;
27        }
28        y = ny;
29        if (match_y[y] == 0) break; // 找到沒配對的右邊點
30    }
31    while (y) match_y[y] = match_y[pre_[y]], y = pre_[y]; // 沿路翻轉
32 }
33 ll km() { // 先填好 n 與 w[1..n][1..n]
34    fill(match_y, match_y + n + 1, 0);
35    fill(ly, ly + n + 1, 0);
36    for (int i = 1; i <= n; i++) lx[i] = *max_element(w[i] + 1, w[i] + n + 1);
37    for (int i = 1; i <= n; i++) km_bfs(i);
38    ll res = 0;
39    for (int i = 1; i <= n; i++) res += w[match_y[i]][i]; // 右 i 配到左
40    match_y[i]
41    return res;

```

## 6.11 Dinic 最大流

**用法:** 反向邊 =  $e^{-1}$ 。附最小割與常見建模。多筆測資呼叫 init(n)。

**時間複雜度:**  $O(V^2 E)$ , 二分圖  $O(E\sqrt{V})$

```

1 // 最大流 Dinic,  $O(V^2 E)$ , 二分圖  $O(E \sqrt{V})$ 。邊 e 的反向邊是  $e^{-1}$ 
2 const int MAXN = 5005, MAXE = 200005; // MAXE = 2 * 邊數
3 const ll INF = 1e18;
4 int n, S, T, ecnt, to[MAXE], lvl[MAXN], it_[MAXN];
5 ll cap[MAXE];
6 vector<int> g[MAXN]; // g[u] = 從 u 出發的邊編號
7 void init(int _n) { n = _n; ecnt = 0; for (int i = 0; i <= n; i++)
8     g[i].clear(); }
9 void add_edge(int u, int v, ll c) { // 無向邊: 第二行的 0 改成 c
10    to[ecnt] = v; cap[ecnt] = c; g[u].push_back(ecnt++);
11    to[ecnt] = u; cap[ecnt] = 0; g[v].push_back(ecnt++);
12 }
13 bool bfs() {
14    fill(lvl, lvl + n + 1, -1);
15    queue<int> q; q.push(S); lvl[S] = 0;
16    while (!q.empty()) {
17        int u = q.front(); q.pop();
18        for (int e : g[u]) if (cap[e] > 0 && lvl[to[e]] == -1) lvl[to[e]] =
19            lvl[u] + 1, q.push(to[e]);
20    }
21    return lvl[T] != -1;
22 }
23 ll dfs(int u, ll f) {
24    if (u == T) return f;
25    for (int &i = it_[u]; i < (int)g[u].size(); i++) { // 當前弧優化: i 是參考
26        int e = g[u][i], v = to[e];
27        if (cap[e] > 0 && lvl[v] == lvl[u] + 1) {
28            ll d = dfs(v, min(f, cap[e]));
29            if (d > 0) { cap[e] -= d; cap[e^{-1}] += d; return d; }
30        }
31    }
32    return 0;
33 }
34 ll max_flow(int s, int t) {
35    S = s; T = t;
36    ll flow = 0;
37    while (bfs()) {
38        fill(it_, it_ + n + 1, 0);
39        while (ll f = dfs(S, INF)) flow += f;
40    }
41    return flow;
42 }
43 /* 建模技巧:
44 - 最小割 = 最大流。割邊: 跑完後從 S 沿 cap>0 走得到的點集合 A, A→非 A 的原始邊
45 - 二分圖匹配: S→左 (1)/左→右 (1)/右→T(1); 每點最多配 k 個→對應邊容量 k
46 - 點有容量: 拆點 v_in→v_out 容量 = 點容量
47 - 多源多匯: 超級源點連所有源點 (容量 INF)

```

```

46 - 最大權閉合子圖 (選 A 必須選 B): S+ 正權點 (w)、負權點 -T(-w)、A+B(INF); 答案
    = 正權和 - 最大流
47 */

```

## 7 樹

### 7.1 Diameter 直徑

**用法:** 兩次 BFS 找最遠點，支援邊權 ( $\geq 0$ )。

**時間複雜度:**  $O(N)$

```

1 // 樹直徑: 從任意點找最遠點 a, 再從 a 找最遠點 b, dist(a,b) 即直徑。邊權須  $\geq 0$ 
2 const int MAXN = 200005;
3 int n, par[MAXN];
4 ll d[MAXN];
5 vector<pair<int, ll>> g[MAXN]; // 無權樹: w = 1
6 int farthest(int s) { // BFS 版 (樹上路徑唯一, 不必 Dijkstra), 避免遞迴太深
7     fill(d, d + n + 1, -1);
8     queue<int> q; q.push(s); d[s] = 0; par[s] = 0;
9     int best = s;
10    while (!q.empty()) {
11        int u = q.front(); q.pop();
12        if (d[u] > d[best]) best = u;
13        for (auto [v, w] : g[u]) if (d[v] == -1) d[v] = d[u] + w, par[v] = u, q.push(v);
14    }
15    return best;
16 }
17 ll diameter(int &a, int &b) {
18     a = farthest(1); b = farthest(a);
19     return d[b]; // 路徑: 從 b 沿 par_ 走回 a
20 }
21 /* [應用]
22 - 每個點的最遠距離 = max(dist(v,a), dist(v,b)) (各跑一次 BFS)
23 - 樹的中心 (最小化最遠距離): 直徑路徑的中點; 從一點廣播到全樹的最短時間 =
    ceil(直徑/2)
24 */

```

### 7.2 Traversal 遍歷

**用法:** 陣列存二元樹的前/中/後序、層序；由前序 + 中序求後序。

**時間複雜度:**  $O(N)$  (重建  $O(N^2)$ )

```

1 // 二元樹用陣列存: lc[u], rc[u] (0 = 沒有子節點), root 通常為 1
2 const int MAXN = 200005;
3 int lc[MAXN], rc[MAXN];
4 vector<int> pre_order, in_order, post_order;
5 void dfs(int u) {
6     if (!u) return;
7     pre_order.push_back(u); // 前序: 根 左 右
8     dfs(lc[u]);
9     in_order.push_back(u); // 中序: 左 根 右 (BST 的中序 = 排序好的)
10    dfs(rc[u]);
11    post_order.push_back(u); // 後序: 左 右 根
12 }
13 vector<int> level_order(int root) { // 層序 (BFS)
14     vector<int> res; queue<int> q; q.push(root);
15     while (!q.empty()) {
16         int u = q.front(); q.pop(); res.push_back(u);
17         if (lc[u]) q.push(lc[u]);
18         if (rc[u]) q.push(rc[u]);
19     }
20     return res;
21 }
22 // 給前序 + 中序, 輸出後序 (字元版)。O(N^2), N 大時用 map 記 in 的位置
23 string pre_in_to_post(const string &pre, const string &in) {
24     if (pre.empty()) return "";
25     char root = pre[0];
26     int k = in.find(root); // 左子樹大小
27     return pre_in_to_post(pre.substr(1, k), in.substr(0, k))
28         + pre_in_to_post(pre.substr(k + 1), in.substr(k + 1)) + root;
29 }
30 // 給後序 + 中序 → 根在後序最後一個, 其餘同理
31 // 一般樹 (不只二元): vector<int> ch[u] 存小孩, DFS 遞迴每個小孩即可

```

### 7.3 LCA 最近共同祖先 (倍增)

**用法:** BFS 建表 (不怕鏈狀樹)。附第  $k$  個祖先、兩點距離、路徑最大邊權。

**時間複雜度:** 預處理  $O(N \log N)$ , 查詢  $O(\log N)$

```

1 // 倍增 LCA: 預處理 O(N log N)、查詢 O(log N)。同時求路徑上的最大邊權
2 const int MAXN = 200005, LOG = 18; // 2^LOG > N
3 int n, up[LOG][MAXN], dep[MAXN];
4 ll mx[LOG][MAXN], distr[MAXN]; // mx[j][v]: v 往上 2^j 條邊中的最大邊權;
    distr: 到根距離
5 vector<pair<int, ll>> g[MAXN];
6 void build(int root) { // BFS 版, 鏈狀樹也不會 stack overflow
7     vector<int> order; order.reserve(n);
8     fill(dep, dep + n + 1, -1);
9     queue<int> q; q.push(root);
10    dep[root] = 0; up[0][root] = root; mx[0][root] = 0; distr[root] = 0;
11    while (!q.empty()) {
12        int u = q.front(); q.pop(); order.push_back(u);
13        for (auto [v, w] : g[u]) if (dep[v] == -1) {
14            dep[v] = dep[u] + 1; up[0][v] = u; mx[0][v] = w; distr[v] =
                distr[u] + w;
15            q.push(v);
16        }
17    }
18    for (int j = 1; j < LOG; j++)
19        for (int v : order) {
20            up[j][v] = up[j - 1][up[j - 1][v]];

```

```

21            mx[j][v] = max(mx[j - 1][v], mx[j - 1][up[j - 1][v]]); // [改]
22            min / sum
23        }
24    int kth_ancestor(int v, int k) { for (int j = 0; j < LOG; j++) if (k >> j &
25        1) v = up[j][v]; return v; }
26    int lca(int u, int v) {
27        if (dep[u] < dep[v]) swap(u, v);
28        u = kth_ancestor(u, dep[u] - dep[v]);
29        if (u == v) return u;
30        for (int j = LOG - 1; j >= 0; j--)
31            if (up[j][u] != up[j][v]) u = up[j][u], v = up[j][v];
32        return up[0][u];
33    }
34    ll path_max(int u, int v) { // u-v 路徑上的最大邊權
35        ll res = 0;
36        if (dep[u] < dep[v]) swap(u, v);
37        for (int j = LOG - 1; j >= 0; j--)
38            if (dep[u] - (1 << j) >= dep[v]) res = max(res, mx[j][u]), u =
                up[j][u];
39        if (u == v) return res;
40        for (int j = LOG - 1; j >= 0; j--)
41            if (up[j][u] != up[j][v]) {
42                res = max({res, mx[j][u], mx[j][v]});
43                u = up[j][u]; v = up[j][v];
44            }
45        return max({res, mx[0][u], mx[0][v]});
46    }
47    // 兩點距離 = distr[u] + distr[v] - 2 * distr[lca(u, v)] (邊數: 用 dep)
48    // u 是否在 v 的子樹內: lca(u, v) == v

```

### 7.4 Kruskal 最小生成樹

**用法:** 邊排序後用 DSU 判環。需要 DisjointSet.cpp。

**時間複雜度:**  $O(E \log E)$

```

1 // 最小生成樹 O(E log E)。需要 DisjointSet.cpp 的 init / unite
2 int n;
3 vector<tuple<ll, int, int>> edges; // {w, u, v}, tuple 會先比 w
4 vector<pair<int, ll>> mst_adj[200005]; // MST 本身 (之後要在樹上做事用)
5 ll kruskal() { // 不連通回傳 -1
6     sort(edges.begin(), edges.end()); // [改] 最大生成樹:
        sort(edges.rbegin(), edges.rend())
7     init(n);
8     for (int i = 0; i <= n; i++) mst_adj[i].clear();
9     ll total = 0; int used = 0;
10    for (auto [w, u, v] : edges)
11        if (unite(u, v)) {
12            total += w; used++;
13            mst_adj[u].push_back({v, w}); mst_adj[v].push_back({u, w});
14        }
15    return used == n - 1 ? total : -1;
16 }
17 /* [變形]
18 - 瓶頸: 「u 到 v 路徑上最大邊最小」= MST 上 u-v 路徑的最大邊 (LCA path_max)
19 - 必須包含某些邊: 先把那些邊 unite 並加總, 再跑 Kruskal
20 - 最小生成「森林」/ 恰好 k 個連通塊: 做到剩 k 塊 (used == n - k) 就停
21 - 稠密圖 (V <= 5000, E ~ V^2): 用 O(V^2) Prim 比較快
22 */

```

### 7.5 嚴格次小生成樹

**用法:** MST + 倍增維護路徑上最大與嚴格次大邊。需要 DisjointSet.cpp。

**時間複雜度:**  $O(E \log V)$

```

1 /* 嚴格次小生成樹 O(E log V)。需要 DisjointSet.cpp
2 想法: 先求 MST; 對每條「非樹邊」(u,v,w), 加進去會形成環,
3 拿掉環上 (= 樹上 u-v 路徑) 最大且 < w 的邊 → 候選 = MST - 那條邊 + w
4 所以要維護路徑上的「最大值」與「嚴格次大值」*/
5 const int LOG = 18; // MAXN 沿用 DisjointSet.cpp 的 (2^LOG > N)
6 const ll INF = 1e18;
7 int n, up[LOG][MAXN], dep[MAXN];
8 ll m1[LOG][MAXN], m2[LOG][MAXN]; // 往上 2^j 條邊的最大、嚴格次大
9 vector<pair<int, ll>> t[MAXN];
10 void merge(ll &a1, ll &a2, ll b1, ll b2) { // 把 (b1,b2) 併進 (a1,a2)
11     for (ll x : {b1, b2}) {
12         if (x > a1) a2 = a1, a1 = x;
13         else if (x < a1 && x > a2) a2 = x;
14     }
15 }
16 ll second_mst(vector<tuple<ll, int, int>> edges) { // 回傳 -1 表示不存在
17     sort(edges.begin(), edges.end());
18     init(n);
19     vector<bool> in_mst(edges.size(), false);
20     ll mst = 0; int used = 0;
21     for (int i = 0; i < (int)edges.size(); i++) {
22         auto [w, u, v] = edges[i];
23         if (unite(u, v)) in_mst[i] = true, mst += w, used++,
            t[u].push_back({v, w}), t[v].push_back({u, w});
24     }
25     if (used != n - 1) return -1;
26     vector<int> order; queue<int> q; q.push(1); // BFS 建倍增表
27     fill(dep, dep + n + 1, -1); dep[1] = 0; up[0][1] = 1; m1[0][1] =
        m2[0][1] = -INF;
28     while (!q.empty()) {
29         int u = q.front(); q.pop(); order.push_back(u);
30         for (auto [v, w] : t[u]) if (dep[v] == -1)
31             dep[v] = dep[u] + 1, up[0][v] = u, m1[0][v] = w, m2[0][v] =
                -INF, q.push(v);
32     }
33     for (int j = 1; j < LOG; j++) for (int v : order) {
34         int mid = up[j - 1][v];
35         up[j][v] = up[j - 1][mid];
36         m1[j][v] = m1[j - 1][v]; m2[j][v] = m2[j - 1][v];

```

```

37         merge_(m1[j][v], m2[j][v], m1[j - 1][mid], m2[j - 1][mid]);
38     }
39     ll best = INF;
40     for (int i = 0; i < (int)edges.size(); i++) {
41         if (in_mst[i]) continue;
42         auto [w, u, v] = edges[i];
43         ll a1 = -INF, a2 = -INF;
44         if (dep[u] < dep[v]) swap(u, v);
45         for (int j = LOG - 1; j >= 0; j--)
46             if (dep[u] - (1 << j) >= dep[v]) merge_(a1, a2, m1[j][u],
47                                                         m2[j][u]), u = up[j][u];
48         if (u != v) {
49             for (int j = LOG - 1; j >= 0; j--) if (up[j][u] != up[j][v]) {
50                 merge_(a1, a2, m1[j][u], m2[j][u]); merge_(a1, a2, m1[j][v],
51                                                         m2[j][v]);
52                 u = up[j][u]; v = up[j][v];
53             }
54             merge_(a1, a2, m1[0][u], m2[0][u]); merge_(a1, a2, m1[0][v],
55                                                         m2[0][v]);
56         }
57         if (a1 < w) best = min(best, mst - a1 + w); // 不要求嚴格:
58         a1 <= w 即可
59         else if (a2 > -INF) best = min(best, mst - a2 + w);
60     }
61     return best == INF ? -1 : best;
62 }
63 // 多筆測資: 記得 t[i].clear()

```

## 8 幾何

### A. 點與直線

點  $(x_0, y_0)$  到直線  $ax + by + c = 0$  的距離  $\frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}$ ; 兩平行線  $ax + by + c_1 = 0$ 、 $ax + by + c_2 = 0$  距離  $\frac{|c_1 - c_2|}{\sqrt{a^2 + b^2}}$ 。

### B. 三角形

頂點  $A(x_1, y_1), B(x_2, y_2), C(x_3, y_3)$ , 對邊長  $a, b, c$ , 面積  $\Delta$ 。

$$\Delta = \frac{1}{2} \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}, \quad \Delta = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$$

重心  $G = \frac{A+B+C}{3}$ ; 內心  $I = \frac{aA+bB+cC}{a+b+c}$ ; 外心  $O$  見 Circle.cpp; 垂心  $H = 3G - 2O$  (重心、外心、垂心共線)。

$$O_x = \frac{1}{4\Delta} \begin{vmatrix} x_1^2 + y_1^2 & y_1 & 1 \\ x_2^2 + y_2^2 & y_2 & 1 \\ x_3^2 + y_3^2 & y_3 & 1 \end{vmatrix}, \quad O_y = \frac{1}{4\Delta} \begin{vmatrix} x_1 & x_1^2 + y_1^2 & 1 \\ x_2 & x_2^2 + y_2^2 & 1 \\ x_3 & x_3^2 + y_3^2 & 1 \end{vmatrix}$$

餘弦定理  $c^2 = a^2 + b^2 - 2ab \cos C$ ; 正弦定理  $\frac{a}{\sin A} = 2R$ ; 內切圓半徑  $r = \frac{\Delta}{s}$ ; 外接圓  $R = \frac{abc}{4\Delta}$ 。

### C. 四邊形

凸四邊形邊長  $a, b, c, d$ ,  $s = \frac{a+b+c+d}{2}$ , 已知  $\angle A + \angle C$ : 面積 =  $\sqrt{(s-a)(s-b)(s-c)(s-d) - abcd \cos^2 \frac{A+C}{2}}$ 。圓內接四邊形時第二項為 0。只知道一個角: 用餘弦定理求對角線, 再用海龍公式算兩個三角形。

### 8.1 點與向量基礎

**用法:** 點用 pair<ld,ld> 表示, 附 + - \* /、內積 dot、外積 cross、sign (處理誤差)。座標全為整數時把 ld 改 ll。

**時間複雜度:**  $O(1)$

```

1 // 點/向量用 pair 表示 (.first = x, .second = y)。座標全是整數時把 ld 改成 ll，
  完全沒有誤差
2 typedef long double ld;
3 typedef pair<ld, ld> P;
4 const ld EPS = 1e-9, PI = acos(-1);
5 int sign(ld x) { return fabs1(x) < EPS ? 0 : (x < 0 ? -1 : 1); } // 浮點數比
  較一律用 sign
6 P operator+(P a, P b) { return {a.first + b.first, a.second + b.second}; }
7 P operator-(P a, P b) { return {a.first - b.first, a.second - b.second}; }
8 P operator*(P a, ld k) { return {a.first * k, a.second * k}; }
9 P operator/(P a, ld k) { return {a.first / k, a.second / k}; }
10 ld dot(P a, P b) { return a.first * b.first + a.second * b.second; } // > 0
  銳角, = 0 垂直, < 0 鈍角
11 ld cross(P a, P b) { return a.first * b.second - a.second * b.first; } // >
  0: b 在 a 的逆時針側 (左邊)
12 ld cross(P o, P a, P b) { return cross(a - o, b - o); } //
  o-a-b: > 0 左轉, < 0 右轉, = 0 共線
13 ld len(P a) { return sqrt1(dot(a, a)); }
14 bool same(P a, P b) { return sign(a.first - b.first) == 0 && sign(a.second -
  b.second) == 0; }
15 P rotate(P a, ld th) { return {a.first * cos1(th) - a.second * sin1(th),
  a.first * sin1(th) + a.second * cos1(th)}; } // 逆時針
16 ld angle(P a) { return atan21(a.second, a.first); } // (-PI, PI]
17 // 極角排序: sort(v.begin(), v.end(), [](P a, P b){ return angle(a) <
  angle(b); });

```

### 8.2 線段相交、直線交點

**用法:** 外積跨立實驗, 可處理端點接觸與共線重疊; 矩形交集面積公式。  
**時間複雜度:**  $O(1)$

```

1 // 需要 point.cpp
2 bool on_segment(P p, P a, P b) { // p 是否在線段 ab 上 (含端點)
3     return sign(cross(a, b, p)) == 0 && sign(dot(a - p, b - p)) <= 0;
4 }
5 // 線段 ab 與 cd 是否相交 (端點接觸、共線重疊)
6 bool seg_intersect(P a, P b, P c, P d) {
7     int d1 = sign(cross(a, b, c)), d2 = sign(cross(a, b, d));
8     int d3 = sign(cross(c, d, a)), d4 = sign(cross(c, d, b));
9     if (d1 * d2 < 0 && d3 * d4 < 0) return true; // 嚴格交叉 (跨立實驗)
10    return on_segment(c, a, b) || on_segment(d, a, b) || on_segment(a, c, d)
11        || on_segment(b, c, d);
12 }
13 // 直線 ab 與直線 cd 的交點 (先確認不平行: sign(cross(b-a, d-c)) != 0)
14 P line_intersection(P a, P b, P c, P d) {
15     ld t = cross(c - a, d - c) / cross(b - a, d - c);
16     return a + (b - a) * t;
17 }
18 /* 軸對齊矩形交集 (左下 (x1,y1) 右上 (x2,y2) 與 (x3,y3)-(x4,y4)):
19    W = max(0, min(x2,x4) - max(x1,x3)), H = max(0, min(y2,y4) - max(y1,y3)),
    面積 = W * H
    (只問「有沒有碰到」: W >= 0 && H >= 0) */

```

### 8.3 點與直線/線段

**用法:** 點到直線、點到線段距離; 投影點、鏡射點。

**時間複雜度:**  $O(1)$

```

1 // 需要 point.cpp
2 ld dist_to_line(P p, P a, P b) { // 點到「直線」ab: 平行四邊形面積 / 底
3     return fabs1(cross(a, b, p)) / len(b - a);
4 }
5 ld dist_to_segment(P p, P a, P b) { // 點到「線段」ab
6     if (same(a, b)) return len(p - a);
7     if (sign(dot(b - a, p - a)) < 0) return len(p - a); // 投影落在 a 外側
8     if (sign(dot(a - b, p - b)) < 0) return len(p - b); // 投影落在 b 外側
9     return dist_to_line(p, a, b);
10 }
11 P projection(P p, P a, P b) { // p 在直線 ab 上的投影點 (垂足)
12     return a + (b - a) * (dot(p - a, b - a) / dot(b - a, b - a));
13 }
14 P reflection(P p, P a, P b) { return projection(p, a, b) * 2 - p; } // 對直線
  ab 的鏡射
15 // 兩線段距離 (不相交時) = 四個「端點到另一線段」距離取 min

```

### 8.4 多邊形面積、點在多邊形內

**用法:** 鞋帶公式 (凹多邊形可); 射線法判斷內/外/邊上; Pick 定理。

**時間複雜度:**  $O(N)$

```

1 // 需要 point.cpp。多邊形頂點依序 (順/逆時針皆可) 存在 vector<P> poly
2 ld polygon_area(const vector<P> &poly) { // 鞋帶公式, 凹多邊形也可
3     ld area = 0; int n = poly.size();
4     for (int i = 0; i < n; i++) area += cross(poly[i], poly[(i + 1) % n]);
5     return fabs1(area) / 2; // 不取絕對值時: > 0 代表逆時針
6 }
7 // 點是否在多邊形內: 1 內部, 0 外部, -1 在邊上 (射線法)
8 int in_polygon(P p, const vector<P> &poly) {
9     int n = poly.size(); bool in = false;
10    for (int i = 0; i < n; i++) {
11        P a = poly[i], b = poly[(i + 1) % n];
12        if (on_segment(p, a, b)) return -1; // 需要
13        Intersection.cpp
14        if ((a.second > p.second) != (b.second > p.second) &&
15            p.first < a.first + (b.first - a.first) * (p.second - a.second)
16            / (b.second - a.second))
17            in = !in;
18    }
19    return in;
20 }
21 /* Pick 定理 (頂點都是格子點): 面積 A = I + B/2 - 1
22    I = 內部格點數, B = 邊上格點數 = Σ gcd(|dx|, |dy|) → I = A - B/2 + 1 */

```

### 8.5 凸包 Convex Hull、旋轉卡尺

**用法:** Andrew 單調鏈, 回傳逆時針凸包; 旋轉卡尺求最遠點對。

**時間複雜度:**  $O(N \log N)$

```

1 // 需要 point.cpp。Andrew 單調鏈, O(N log N), 回傳逆時針凸包 (不含共線點)
2 vector<P> convex_hull(vector<P> pts) {
3     sort(pts.begin(), pts.end()); // pair 預設先比 x 再比 y
4     pts.erase(unique(pts.begin(), pts.end(), pts.end()));
5     int n = pts.size();
6     if (n <= 2) return pts;
7     vector<P> h(2 * n);
8     int m = 0;
9     for (int i = 0; i < n; i++) { // 下凸包 (左 → 右)
10        while (m >= 2 && sign(cross(h[m - 2], h[m - 1], pts[i])) <= 0) m--;
11        // [改] 保留共線點: < 0
12        h[m++] = pts[i];
13    }
14    for (int i = n - 2, t = m + 1; i >= 0; i--) { // 上凸包 (右 → 左)
15        while (m >= 2 && sign(cross(h[m - 2], h[m - 1], pts[i])) <= 0) m--;
16        h[m++] = pts[i];
17    }
18    h.resize(m - 1); // 起點被加了兩次
19    return h;
20 }
21 // 旋轉卡尺: 凸包直徑 (最遠點對距離的平方) * h 為逆時針凸包
22 ld farthest_pair(const vector<P> &h) {
23     int n = h.size();

```

```

23     if (n == 1) return 0;
24     if (n == 2) return dot(h[0] - h[1], h[0] - h[1]);
25     ld best = 0;
26     for (int i = 0, j = 1; i < n; i++) {
27         while (cross(h[i], h[(i + 1) % n], h[(j + 1) % n]) > cross(h[i], h[(i + 1) % n], h[j])) j = (j + 1) % n;
28         best = max({best, dot(h[i] - h[j], h[i] - h[j]), dot(h[(i + 1) % n] - h[j], h[(i + 1) % n] - h[j])});
29     }
30     return best;
31 }
32 // 凸包周長 = Σ len(h[i+1] - h[i]); 面積用 polygon_area(h)

```

## 8.6 圓

**用法:** 三點求外接圓、直線與圓的交點、兩圓關係。

**時間複雜度:**  $O(1)$

```

1 // 需要 point.cpp。圓用 {圓心 P, 半徑 ld} 表示
2 // 三點決定的圓 (三角形外接圓)。三點共線時半徑回傳 -1
3 pair<P, ld> circumcircle(P a, P b, P c) {
4     P ab = b - a, ac = c - a;
5     ld d = 2 * cross(ab, ac);
6     if (sign(d) == 0) return {{0, 0}, -1};
7     ld b2 = dot(ab, ab), c2 = dot(ac, ac);
8     P o = a + P({ac.second * b2 - ab.second * c2} / d, {ab.first * c2 - ac.first * b2} / d);
9     return {o, len(o - a)};
10 }
11 // 直線 ab 與圓 (o, r) 的交點 (0-2 個)
12 vector<P> circle_line(P o, ld r, P a, P b) {
13     P h = projection(o, a, b);
14     ld d2 = r * r - dot(h - o, h - o);
15     if (sign(d2) < 0) return {};
16     P dir = (b - a) / len(b - a);
17     ld s = sqrt1(max(0, d2));
18     if (sign(s) == 0) return {h};
19     return {h - dir * s, h + dir * s};
20 }
21 // 兩圓關係 (d = 圓心距): d > r1+r2 相離; d == r1+r2 外切; |r1-r2| < d < r1+r2 相交;
22 // d == |r1-r2| 內切; d < |r1-r2| 內含

```

## 9 DP

**DP 四步驟:** 1. 定義狀態: 寫清楚  $dp[i][j]$  代表什麼 (題目多一個條件 = 多一維)。2. 轉移: 「最後一步」是什麼? 由哪些較小的狀態來? 3. 初始值:  $dp[0]$ 、不可能的狀態設  $\pm INF$ 。4. 計算順序: 確保用到的狀態都已算好 (通常由小到大)。答案在哪一格?

**常見狀態:** 前  $i$  個 (線性)、區間  $[l, r]$  (區間)、拓撲序 (DAG)。

### 9.1 背包問題

**用法:** 0/1 (含回溯)、二維費用 (兩種資源同時限制, 例: 時間 + 雞蛋)、完全、多重 (二進位拆分)、分組; 子集和和 bitset、方法數、容量很大時的變形。

**時間複雜度:**  $O(NW)$

```

1 // n 個物品 (1-based), 重量 w[i]、價值 v[i], 容量 W
2 const int MAXN = 1005, MAXW = 100005;
3 int n, W, w[MAXN], v[MAXN];
4 ll dp[MAXNW];
5 ll dp2[105][10005]; // 二維版很吃記憶體 (N*W*8 bytes), 只要回溯時用
6
7 // 1. 0/1 背包 二維 (要回溯選了哪些時用)。dp2[i][j] = 前 i 個物品、容量 j 的最大價值
8 void knap01_2d() {
9     for (int i = 1; i <= n; i++)
10         for (int j = 0; j <= W; j++) {
11             dp2[i][j] = dp2[i - 1][j];
12             // 不拿 i
13             if (j >= w[i]) dp2[i][j] = max(dp2[i][j], dp2[i - 1][j - w[i]] + v[i]); // 拿 i
14         }
15     vector<int> chosen; // 回溯: 和上一列一樣就是沒拿
16     for (int i = n, j = W; i >= 1; i--)
17         if (dp2[i][j] != dp2[i - 1][j]) chosen.push_back(i), j -= w[i];
18 }
19 // 2. 0/1 背包 一維 (滾動陣列): j 必須「由大到小」, 否則同一物品會被拿多次
20 void knap01() {
21     fill(dp, dp + W + 1, 0); // [改] 恰好裝滿: dp[0]=0, 其餘 -INF
22     for (int i = 1; i <= n; i++)
23         for (int j = W; j >= w[i]; j--) dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
24 }
25 // 2b. 二維費用 0/1 背包: 每個物品同時消耗兩種資源 (例: 時間 t[i]、雞蛋 e[i]), 上限 T、E
26 const int MAXT = 505, MAXE = 505;
27 int T, E, t[MAXN], e[MAXN];
28 ll dp2d[MAXN][MAXE];
29 void knap01_2cost() {
30     for (int j = 0; j <= T; j++) fill(dp2d[j], dp2d[j] + E + 1, 0);
31     for (int i = 1; i <= n; i++)
32         for (int j = T; j >= t[i]; j--) // 時間由大到小
33             for (int k = E; k >= e[i]; k--) // 雞蛋由大到小
34                 dp2d[j][k] = max(dp2d[j][k], dp2d[j - t[i]][k - e[i]] + v[i]);
35 }
36 // 答案 dp2d[T][E]。[改] 每個物品可無限拿: 兩層都改成由小到大
37 // [改] 求「最多能做幾個»: v[i] = 1

```

```

37 }
38 // 3. 完全背包 (每種無限個): j「由小到大」
39 void knap_unbounded() {
40     fill(dp, dp + W + 1, 0);
41     for (int i = 1; i <= n; i++)
42         for (int j = w[i]; j <= W; j++) dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
43 }
44 // 4. 多重背包 (第 i 種有 c[i] 個): 二進位拆分成 1, 2, 4, ..., 剩餘 → 變 0/1 背包,
45 // 0(W Σ log c)
46 int c[MAXN];
47 void knap_bounded() {
48     fill(dp, dp + W + 1, 0);
49     for (int i = 1; i <= n; i++)
50         for (int k = 1, left = c[i]; left > 0; k <= 1) {
51             int take = min(k, left); left -= take;
52             int ww = w[i] * take; ll vv = (ll)v[i] * take;
53             for (int j = W; j >= ww; j--) dp[j] = max(dp[j], dp[j - ww] + vv);
54         }
55 }
56 // 5. 分組背包 (每組最多選一個): 容量迴圈在外 (由大到小), 組內物品在內
57 vector<vector<pair<int, int>>> groups; // groups[g] = {(重量, 價值)}
58 void knap_group() {
59     fill(dp, dp + W + 1, 0);
60     for (auto &grp : groups)
61         for (int j = W; j >= 0; j--)
62             for (auto [gw, gv] : grp) if (j >= gw) dp[j] = max(dp[j], dp[j - gw] + gv);
63 }
64 // 6. 子集和 / 能否剛好湊出 / 分成兩堆等重: bitset<MAXW> b; b[0]=1; 每個 x: b |= b << x;
65 // 能湊出 j <=> b[j]。等重兩堆: 總和偶數且 b[sum/2]
66 // 7. 方法數: dp[0]=1, max 改成 +=
67 // 8. 容量很大 (1e9) 但價值小: 改成 dp[價值] = 最小重量, 最後找 dp[val] <= W 的最大 val
68 // 9. 多人各自有容量 W_i、共用同一批商品: 跑一次 dp, 答案 Σ dp[W_i] (dp 初始全 0 時已是單調) */

```

### 9.2 分錢幣問題

**用法:** 完全背包的變形: 組合數 (外層硬幣)、排列數 (外層金額)、最少硬幣數。

**時間複雜度:**  $O(\text{硬幣數} \times K)$

```

1 // 湊出金額 k (硬幣無限使用)。方法數成長很快 → 取模或 long long
2 const int MAXK = 1000005;
3 const ll MOD = 1e9 + 7, INF = 1e18;
4 ll dp[MAXK];
5 // 1. 組合數 (2+3 和 3+2 算同一種): 外層「硬幣」、內層「金額」
6 ll count_combination(const vector<int> &coins, int k) {
7     fill(dp, dp + k + 1, 0); dp[0] = 1;
8     for (int c : coins)
9         for (int i = c; i <= k; i++) dp[i] = (dp[i] + dp[i - c]) % MOD;
10    return dp[k];
11 }
12 // 2. 排列數 (順序不同算不同): 外層「金額」、內層「硬幣」
13 ll count_permutation(const vector<int> &coins, int k) {
14     fill(dp, dp + k + 1, 0); dp[0] = 1;
15     for (int i = 1; i <= k; i++)
16         for (int c : coins) if (i >= c) dp[i] = (dp[i] + dp[i - c]) % MOD;
17    return dp[k];
18 }
19 // 3. 最少硬幣數 (湊不出回傳 -1)
20 ll min_coins(const vector<int> &coins, int k) {
21     fill(dp, dp + k + 1, INF); dp[0] = 0;
22     for (int c : coins)
23         for (int i = c; i <= k; i++) if (dp[i - c] != INF) dp[i] = min(dp[i], dp[i - c] + 1);
24     return dp[k] == INF ? -1 : dp[k];
25 }
26 // 多筆詢問同一組硬幣: 在 main 開頭對最大金額建一次表, 每筆直接查 dp[k]
27 // 每種硬幣只能用一次 → 內層改成由大到小 (變 0/1 背包)

```

### 9.3 LCS 最長共同子序列、編輯距離

**用法:** 二維 DP + 回溯出字串。

**時間複雜度:**  $O(NM)$

```

1 // dp[i][j] = a 前 i 個字元、b 前 j 個字元 的答案。index 0 = 空字串
2 const int MAXN = 3005; // 5000x5000 int = 100MB 太大, 注意記憶體
3 int dp[MAXN][MAXN];
4 string lcs(const string &a, const string &b) { // 最長共同子序列 (回傳字串)
5     int n = a.size(), m = b.size();
6     for (int i = 0; i <= n; i++)
7         for (int j = 0; j <= m; j++) {
8             if (i == 0 || j == 0) dp[i][j] = 0;
9             else if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
10            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
11        }
12    string res;
13    for (int i = n, j = m; i > 0 && j > 0;) { // 回溯
14        if (a[i - 1] == b[j - 1]) res += a[i - 1], i--, j--;
15        else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
16        else j--;
17    }
18    reverse(res.begin(), res.end());
19    return res; // 長度 = dp[n][m]
20 }
21 int edit_distance(const string &a, const string &b) { // 插入/刪除/取代 各 1 次
22     int n = a.size(), m = b.size();
23     for (int i = 0; i <= n; i++)
24         for (int j = 0; j <= m; j++) {

```

```

25         if (i == 0 || j == 0) { dp[i][j] = i + j; continue; } // 全部插入/刪除
26         dp[i][j] = min({dp[i - 1][j] + 1, // 刪
27             dp[i][j - 1] + 1, // 插
28             dp[i - 1][j - 1] + (a[i - 1] != b[j - 1])}); // 取代/相同
29     }
30     return dp[n][m]; // 輸出過程：從 (n,m) 往回看是哪一項轉移來的
31 }
32 /* - 最長迴文子序列 = lcs(s, reverse(s)).size(); 最少插入幾個字變迴文 = n - 它
33    - 最長共同「子字串」(要連續)：相同時 dp = dp[i-1][j-1]+1, 不同時 dp = 0, 答案取全部最大
34    - 只有插入/刪除 (沒有取代) 的距離 = n + m - 2 * LCS
35    - 空間不夠：只保留兩列 (dp[2][MAXN]), 用 i & 1 */

```

## 9.4 LIS 最長遞增子序列

**用法：**嚴格用 lower\_bound、非嚴格用 upper\_bound。附輸出真正的 LIS 與常見題型。

**時間複雜度：** $O(N \log N)$

```

1 // LIS O(N log N)。tail[k] = 長度 k+1 的遞增子序列「最小的結尾」
2 // 嚴格遞增：lower_bound; 非嚴格 (可相等)：upper_bound
3 int lis_length(const vector<int> &a) {
4     vector<int> tail;
5     for (int x : a) {
6         auto it = lower_bound(tail.begin(), tail.end(), x); // [改] 非嚴格：
7             upper_bound
8         if (it == tail.end()) tail.push_back(x); else *it = x;
9     }
10    return tail.size(); // 注意 tail 的內容不一定是真正的 LIS
11 // 要輸出真正的 LIS：記錄每個元素接在誰後面
12 vector<int> lis_sequence(const vector<int> &a) {
13     int n = a.size();
14     vector<int> tail_val, tail_idx, prv(n, -1);
15     for (int i = 0; i < n; i++) {
16         int k = lower_bound(tail_val.begin(), tail_val.end(), a[i]) -
17             tail_val.begin();
18         if (k == (int)tail_val.size()) tail_val.push_back(a[i]),
19             tail_idx.push_back(i);
20         else tail_val[k] = a[i], tail_idx[k] = i;
21         prv[i] = k ? tail_idx[k - 1] : -1;
22     }
23     vector<int> res;
24     for (int i = tail_idx.empty() ? -1 : tail_idx.back(); i != -1; i =
25         prv[i]) res.push_back(a[i]);
26     reverse(res.begin(), res.end());
27     return res;
28 }
29 // LDS：把每個數取負再做 LIS (嚴格遞減 → 對 -a 用 lower_bound; 非嚴格遞減 →
30     upper_bound)
31 /* [題型]
32 - 俄羅斯娃娃/信封 (w,h 都要嚴格大)：依 w 遞增排、w 相同時 h 遞減排，再對 h 做嚴格 LIS
33 - 橋不交叉：依南岸排序，對北岸做 LIS
34 - 最少幾個「遞減序列」覆蓋全部 (攔截飛彈) = 嚴格遞增 LIS 長度 (Dilworth)
35 - 先增後減 (Bitonic)：L[i] = 以 i 結尾的 LIS、R[i] = 以 i 開頭的 LDS，
36     max(L[i]+R[i]-1)
37 - 以 i 結尾的 LIS 長度 = 插入位置 k + 1
38 - 最少刪幾個變遞增 = n - LIS; 最少改幾個變嚴格遞增：對 a[i]-i 做非嚴格 LIS */

```

## 9.5 LCS 轉 LIS

**用法：** $B$  的每個數換成它在  $A$  中的位置，求嚴格 LIS = LCS。適合  $A, B$  為排列時。

**時間複雜度：** $O(K \log K)$ ， $K$  為配對數

```

1 // 兩序列 LCS，當「每個值在 A 中出現次數很少」(例如 A、B 都是排列)時  $O(K \log K)$ 
2 // 把 B 中每個數換成它在 A 中的位置 (由大到小放)，對新序列求嚴格 LIS = LCS 長度
3 int lcs_by_lis(const vector<int> &a, const vector<int> &b) {
4     unordered_map<int, vector<int>> pos;
5     for (int i = 0; i < (int)a.size(); i++) pos[a[i]].push_back(i);
6     vector<int> seq;
7     for (int x : b) {
8         auto it = pos.find(x);
9         if (it == pos.end()) continue;
10        for (int k = (int)it->second.size() - 1; k >= 0; k--)
11            seq.push_back(it->second[k]); // 反向避免同一個 B 元素配到兩次
12    }
13    vector<int> tail;
14    for (int x : seq) {
15        auto p = lower_bound(tail.begin(), tail.end(), x);
16        if (p == tail.end()) tail.push_back(x); else *p = x;
17    }
18    return tail.size();

```

## 9.6 區間 DP

**用法：**由短區間推長區間。合併石頭通用模板、最長迴文子序列、變迴文的最少編輯 (含輸出)。

**時間複雜度：** $O(N^3) / O(N^2)$

```

1 /* 區間 DP: dp[l][r] 由「更短的區間」算出來 → 依長度由短到長 (或 1 由大到小、r
2     由小到大)
3     通用模板 (合併石頭、矩陣鏈乘、切木棍)：枚舉分割點 k
4     dp[l][r] = min_{l<=k<r} dp[l][k] + dp[k+1][r] + cost(l, r)     $O(N^3)$ ， $N$ 
5     <= 500 */
6 const int MAXN = 505;
7 const ll INF = 1e18;

```

```

6 int n;
7 ll a[MAXN], pre[MAXN], dp[MAXN][MAXN];
8 ll merge_stones() { // 相鄰兩堆合併，代價 = 兩堆總和，求最小總代價 (a 為
9     1-based)
10    for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i], dp[i][i] = 0;
11    for (int len = 2; len <= n; len++)
12        for (int l = 1, r = len; r <= n; l++, r++) {
13            dp[l][r] = INF;
14            for (int k = l; k < r; k++) dp[l][r] = min(dp[l][r], dp[l][k] +
15                dp[k + 1][r]);
16            dp[l][r] += pre[r] - pre[l - 1]; // [改] cost(l, r)
17        }
18    return dp[1][n]; // 環狀：複製一份接在後面 (長度 2n)，答案 min
19        dp[i][i+n-1]
20 }
21 // 最長迴文子序列 (LPS)，0-based
22 int lps(const string &s) {
23     int m = s.size();
24     vector<vector<int>> f(m, vector<int>(m, 0));
25     for (int i = m - 1; i >= 0; i--) {
26         f[i][i] = 1;
27         for (int j = i + 1; j < m; j++)
28             f[i][j] = s[i] == s[j] ? f[i + 1][j - 1] + 2 : max(f[i + 1][j],
29                 f[i][j - 1]);
30     }
31     return m ? f[0][m - 1] : 0;
32 }
33 // 變成迴文的最少編輯 (插入/刪除/取代都算 1)，並輸出字典序最小的結果字串
34 pair<int, string> palindrome_edit(const string &s) {
35     int m = s.size();
36     if (!m) return {0, ""};
37     vector<vector<int>> f(m + 1, vector<int>(m + 1, 0));
38     for (int i = m - 1; i >= 0; i--)
39         for (int j = i + 1; j < m; j++)
40             f[i][j] = s[i] == s[j] ? f[i + 1][j - 1] : min({f[i + 1][j],
41                 f[i][j - 1], f[i + 1][j - 1] + 1});
42     string L, R; int i = 0, j = m - 1;
43     while (i < j) {
44         if (s[i] == s[j]) { L += s[i]; R += s[j]; i++; j--; continue; }
45         int best = 256, ch = 0; // 1 取代, 2 處理左, 3 處理右; 同分取字典序小的
46         字元
47         if (f[i + 1][j - 1] + 1 == f[i][j] && min(s[i], s[j]) < best) best =
48             min(s[i], s[j]), ch = 1;
49         if (f[i + 1][j] + 1 == f[i][j] && s[i] < best) best = s[i], ch = 2;
50         if (f[i][j - 1] + 1 == f[i][j] && s[j] < best) best = s[j], ch = 3;
51         L += (char)best; R += (char)best;
52         if (ch == 1) i++, j--; else if (ch == 2) i++; else j--;
53     }
54     if (i == j) L += s[i];
55     reverse(R.begin(), R.end());
56     return {f[0][m - 1], L + R};
57 }
58 /* 迴文子序列「總數」(位置不同算不同):
59    s[i]==s[j]: f = f[i+1][j] + f[i][j-1] + 1; 否則 f = f[i+1][j] + f[i][j-1]
60    - f[i+1][j-1] */

```

## 9.7 加權區間排程 (工作有獎勵)

**用法：**選互不衝突的工作使獎勵和最大，並回溯輸出選了哪些。依結束時間排序 + 二分搜前一個不衝突的工作。TOUCH\_OK 決定「端點相接」算不算衝突。

**時間複雜度：** $O(N \log N)$

```

1 /* 加權區間排程：n 個工作 {開始 s, 結束 e, 獎勵 r}，選出互不衝突的工作使獎勵和最
2     大，並輸出選了哪些
3     (沒有獎勵、只求最多幾個 → Greedy「最多不重疊區間」即可)
4     作法：依結束時間排序。dp[i] = 只看前 i 個工作 (排序後) 的最大獎勵
5     dp[i] = max(dp[i-1], 不選第 i 個
6         dp[p[i]] + r[i]) 選第 i 個, p[i] = 排序後「結束時間不衝突」
7         的工作有幾個
8     p[i] 用二分搜在排好序的結束時間 ed[] 中找。O(N log N) */
9 const int MAXN = 200005;
10 int n, id_[MAXN], p[MAXN];
11 ll s[MAXN], e[MAXN], r[MAXN], ed[MAXN], dp[MAXN];
12 bool TOUCH_OK = true; // [改] true：端點相接不算衝突 (e == 下一個的 s 可以接著
13     做)
14 // false: 端點相接也算衝突 (必須 e < 下一個的 s)
15 ll weighted_interval(vector<int> &chosen) { // 輸入 s[1..n], e[1..n],
16     r[1..n]; chosen 回傳原始編號 (1-based)
17    for (int i = 1; i <= n; i++) id_[i] = i;
18    sort(id_ + 1, id_ + n + 1, [](int a, int b) { return make_pair(e[a],
19        s[a]) < make_pair(e[b], s[b]); }); // 依結束時間排序 (同時結束再比開始)
20    for (int i = 1; i <= n; i++) ed[i] = e[id_[i]];
21    dp[0] = 0;
22    for (int i = 1; i <= n; i++) {
23        ll st = s[id_[i]];
24        if (TOUCH_OK) p[i] = upper_bound(ed + 1, ed + i, st) - (ed + 1); //
25            結束 <= st 的個數
26        else p[i] = lower_bound(ed + 1, ed + i, st) - (ed + 1); //
27            結束 < st 的個數
28        dp[i] = max(dp[i - 1], dp[p[i]] + r[id_[i]]);
29    }
30    chosen.clear(); // 回溯：dp[i] == dp[i-1] 代表沒選第 i 個，否則選了它並跳
31        到 p[i]
32    for (int i = n; i > 0; i--) {
33        if (dp[i] == dp[i - 1]) i--;
34        else chosen.push_back(id_[i]), i = p[i];
35    }
36    sort(chosen.begin(), chosen.end()); // [改] 要依時間順序輸出：改成依 s 排
37        序
38    return dp[n];
39 }
40 int main() { // 輸入：n，接著 n 行 s e r; 輸出最大獎勵與選的工作編號
41    cin >> n;

```

```
33     for (int i = 1; i <= n; i++) cin >> s[i] >> e[i] >> r[i];
34     vector<int> chosen;
35     cout << weighted_interval(chosen) << '\n';
36     for (int i = 0; i < (int)chosen.size(); i++) cout << chosen[i] << " \n"[i
37         + 1 == (int)chosen.size()];
38     return 0;
39 }
40 /* 範例: (1,3,5) (2,4,6) (3,5,5) (4,6,4)
41    TOUCH_OK = true : 10 (選 1,3 或 2,4)    TOUCH_OK = false: 9 (選 1,4)
42    [變形]
43    - 時間是「第幾天」且 [s, e] 兩天都要佔用 (整數閉區間) → TOUCH_OK = false
44    - 求「最多幾個工作」→ r 全設 1 (也可直接用 Greedy)
45    - 時間很大 (1e9) 也沒關係, 只用到排序和二分搜, 不用開時間大小的陣列 */
```

## 9.8 Kadane 最大子陣列和

**用法:** 全為負數時回傳最大的負數。附二維、環狀等變形。

**時間複雜度:**  $O(N)$

```
1 // 最大連續子陣列和  $O(N)$ 。全部是負數時回傳最大的那個負數
2 ll kadane(const vector<ll> &a) {
3     ll best = LLONG_MIN, cur = 0;
4     for (ll x : a) {
5         cur = max(x, cur + x); // 接在前面 vs 從自己重新開始
6         best = max(best, cur);
7     }
8     return best; // 允許空陣列 (答案至少 0): best 初始 0
9 }
10 /* [變形]
11 - 最大子矩形和 (N,M <= 300): 枚舉上下列 r1..r2, 把每行壓成一個數 (行前綴和), 對
12   它做 kadane,  $O(N^2 M)$ 
13 - 環狀陣列: max(一般 kadane, 總和 - 最小子陣列和); 全負時只取一般 kadane
14 - 長度至少 K: 前綴和 pre, ans = max(pre[i] - min(pre[0..i-K]))
15 - 最大子陣列乘積: 同時維護 cur_max * cur_min (負負得正) */
```

## 9.9 跳格子 (路徑還原)

**用法:** 一次最多跳  $K$  格, 最少步數並輸出路徑。

**時間複雜度:**  $O(NK)$

```
1 /* PUPC 2025 River Crossing: v[i]=1 可踩、0 不可踩, 一次最多跳 K 格,
2   求從 0 到 n-1 最少跳幾次並輸出路徑 (同步數時選 index 較小的前一格)。 $O(N*K)$ 
3   */
4 const int MAXN = 200005, INF = 0x3f3f3f3f;
5 int n, K = 3, v[MAXN], f[MAXN], pre[MAXN]; // [改] K
6 void solve() {
7     fill(f, f + n, INF); fill(pre, pre + n, -1);
8     f[0] = 0;
9     for (int i = 1; i < n; i++) {
10         if (!v[i]) continue;
11         for (int j = 1; j <= K && i - j >= 0; j++)
12             if (v[i - j] && f[i - j] != INF && f[i - j] + 1 <= f[i]) // <= :
13                 同步數時換成更小的 index
14                 f[i] = f[i - j] + 1, pre[i] = i - j;
15     }
16     if (f[n - 1] == INF) { cout << -1 << '\n'; return; }
17     vector<int> path;
18     for (int i = n - 1; i != -1; i = pre[i]) path.push_back(i);
19     reverse(path.begin(), path.end());
20     cout << f[n - 1] << '\n';
21     for (int i = 0; i < (int)path.size(); i++) cout << path[i] << " \n"[i + 1
22         == (int)path.size()];
23 }
24 // 若 K 很大: f[i] = min(f[i-K..i-1]) + 1 → 用單調隊列,  $O(N)$ 
25 // 不用輸出路徑時: 用 Greedy「間隔跳躍」 $O(N)$ 
```

## 10 Greedy

**Greedy 不確定對不對時:** 用「交換論證」(相鄰兩個交換會不會更好) 推出排序方式; 或寫暴力對拍。

### 10.1 區間覆蓋 (灑水器)

**用法:** 用最少區間蓋滿  $[0, L]$ : 依左端排序, 每次選能延伸最遠的。

**時間複雜度:**  $O(N \log N)$

```
1 /* 區間覆蓋: 用最少的區間蓋滿  $[0, L]$ 。灑水器版 (UVa 10382): 草地寬 W,
2   灑水器在 pos、半徑 r, 能蓋到的範圍是  $pos \pm \sqrt{r^2 - (W/2)^2}$ 
3   作法: 依左端排序, 每次在「左端 <= 目前已蓋到的位置」的區間中選右端最遠的 */
4 const double eps = 1e-9;
5 int min_cover(vector<pair<double, double>> seg, double L) { // seg = {左,
6     右}, 無解回傳 -1
7     sort(seg.begin(), seg.end());
8     int cnt = 0, i = 0, n = seg.size();
9     double reach = 0; // [改] 起點
10    while (reach < L - eps) {
11        double best = reach;
12        while (i < n && seg[i].first <= reach + eps) best = max(best,
13            seg[i].second), i++;
14        if (best <= reach + eps) return -1; // 接不上
15        reach = best; cnt++; // 要輸出選了哪些: 記錄 best 是哪個區
16        間
17    }
18    return cnt;
19 }
20 // 灑水器轉區間: if (r * 2 <= W) 跳過; half = sqrt(r*r - W*W/4.0);
21 seg.push_back({pos - half, pos + half});
22 // 整數點版本 (要蓋住點 1..L, [1,3] 與 [4,5] 算接起來): 條件改成 seg[i].first
23 <= reach + 1
```

### 10.2 區間選點

**用法:** 最少幾個點讓每個區間都含點: 依右端排序。

**時間複雜度:**  $O(N \log N)$

```
1 // 區間選點: 最少選幾個點, 使每個區間 [l, r] 內至少有一個點。依「右端」排序, 點盡
2   量放右端
3 int min_points(vector<pair<int, int>> seg) { // seg = {l, r}
4     sort(seg.begin(), seg.end(), [](auto &a, auto &b) { return a.second <
5         b.second; });
6     int cnt = 0; ll last = LLONG_MIN;
7     for (auto [l, r] : seg)
8         if (l > last) cnt++, last = r; // 這個區間還沒被蓋到 → 在它的右端放一個
9         點
10    return cnt;
11 }
12 // 答案也等於「最多能選幾個互不相交的區間」(同一個 greedy)
```

### 10.3 最多不重疊區間

**用法:** 依結束時間排序, 能選就選。最少刪除數 =  $N -$  答案。

**時間複雜度:**  $O(N \log N)$

```
1 // 活動選擇: 找到最多選幾個互不重疊的區間 (或 = n - 最少刪除數)。依「結束時間」排
2   序, 能選就選
3 int max_non_overlap(vector<pair<int, int>> seg) { // {start, end}, [s, e] 半
4   開區間
5     sort(seg.begin(), seg.end(), [](auto &a, auto &b) { return a.second <
6         b.second; });
7     int cnt = 0; ll cur_end = LLONG_MIN; // 一開始什麼都沒選; 第一個 (最早結束
8   的) 會在迴圈裡被選到
9     for (auto [s, e] : seg)
10         if (s >= cur_end) cnt++, cur_end = e; // [改] 閉區間 (端點碰到也算重
11         疊): s > cur_end
12    return cnt; // cnt 直接是答案
13 }
14 // 另一種寫法 (CSES Movie Festival): cnt = 1, cur_end = 第一個的 end, 迴圈「從
15   第二個開始」
16 // 兩種起始值不能混用: cnt = 1 配 LLONG_MIN 會把第一個算兩次
17 // 有權重 (每個活動有獎勵) → 不能 Greedy, 看 DP 的「加權區間排序」
18   (WeightedInterval.cpp)
```

### 10.4 最少會議室

**用法:** 依開始時間排序, heap 存使用中房間的結束時間。

**時間複雜度:**  $O(N \log N)$

```
1 // 最少會議室: 依開始時間排序, heap 存「每間使用中房間的結束時間」
2 int min_rooms(vector<pair<int, int>> seg) { // {start, end}
3     sort(seg.begin(), seg.end());
4     priority_queue<int, vector<int>, greater<int>> pq;
5     for (auto [s, e] : seg) {
6         if (!pq.empty() && pq.top() <= s) pq.pop(); // [改] 閉區間 (結束當下不
7         能接): <
8         pq.push(e);
9     }
10    return pq.size();
11 }
12 // 要輸出每個會議分到哪間: heap 改存 {結束時間, 房號}, pop 出來的房號給目前的會
13   議
14 // 答案也等於「同一時間最多重疊幾個區間」→ 也可以用 SweepLine
```

### 10.5 掃描線: 最大重疊數

**用法:** 開始 +1、結束 -1。同時時間的處理順序取決於開/閉區間, 見註解。

**時間複雜度:**  $O(N \log N)$

```
1 // 掃描線: 同一時刻最多重疊幾個區間。開始 +1、結束 -1, 依時間排序
2 int max_overlap(const vector<pair<int, int>> &seg) {
3     vector<pair<int, int>> ev;
4     for (auto [s, e] : seg) ev.push_back({s, +1}), ev.push_back({e, -1});
5     sort(ev.begin(), ev.end()); // 同時時間 -1 排在 +1 前面
6     int cur = 0, best = 0;
7     for (auto [t, d] : ev) cur += d, best = max(best, cur);
8     return best;
9 }
10 /* 同一時間點的順序很重要 (和題目定義有關, 看清楚!):
11 - 半開區間 [s, e): e 時刻已離開 → 先 -1 再 +1 (上面的預設)
12 - 閉區間 [s, e] : e 時刻還在 → 先 +1 再 -1 → 把結束事件改成 {e + 1, -1}
13   (整數時間)
14 聯集總長度: 掃描時 cur > 0 的區段長度加總
15 二維 (矩形面積聯集): x 方向掃描線 + 線段樹維護 y 方向被覆蓋的長度 */
```

### 10.6 最小化最大延遲

**用法:** 依期限由早到晚做。附 SJF、加權完成時間、交換論證。

**時間複雜度:**  $O(N \log N)$

```
1 // 最小化最大延遲: 每個工作 {耗時, 期限}, 一台機器依序做。依「期限」由早到晚做最好
2 ll min_max_lateness(vector<pair<ll, ll>> jobs) { // {time, deadline}
3     sort(jobs.begin(), jobs.end(), [](auto &a, auto &b) { return a.second <
4         b.second; });
5     ll t = 0, ans = 0;
6     for (auto [len, dl] : jobs) t += len, ans = max(ans, t - dl);
7     return ans; // 0 代表全部準時
8 }
9 // 最小化「總完成時間」(或平均等待時間): 依耗時由短到長 (SJF)
10 // 加權版  $\sum w_i * C_i$ : 依 time / weight 由小到大 (比較時用交叉相乘避免浮點)
11 // 交換論證: 相鄰兩工作 i, j 交換前後的代價比較 → 得到排序的比較函式 (自己寫 cmp
12   的萬用方法)
```

## 10.7 有期限的單位工作排程

**用法:** 最小化逾期懲罰。期限  $\leq 10^6$  用 DSU 找空位；期限很大用依期限排序 + min-heap。

**時間複雜度:**  $O(N \log N)$

```
1 // 每個工作耗時 t、有期限 d 與懲罰 p (或利潤)。逾期要付懲罰，求最小總懲罰 (= 最
  大化準時的利潤)
2 // 方法一：懲罰大的先排，放在「期限內最晚的空閒時間」，用 DSU 找空位。近乎  $O(N)$ ，
  期限  $\leq 1e6$ 
3 ll min_penalty_dsu(vector<pair<int, int>> jobs) { // {deadline, penalty}
4     sort(jobs.begin(), jobs.end(), [](auto &a, auto &b) { return a.second >
      b.second; });
5     int maxd = 0;
6     for (auto [d, p] : jobs) maxd = max(maxd, d);
7     vector<int> f(maxd + 1);
8     iota(f.begin(), f.end(), 0);
9     function<int(int)> find = [&](int x) { return f[x] == x ? x : f[x] =
      find(f[x]); };
10    ll pen = 0;
11    for (auto [d, p] : jobs) {
12        int slot = find(min(d, maxd));
13        if (slot > 0) f[slot] = slot - 1; // 佔用 slot，之後找到 slot 會跳到更
      早
14        else pen += p; // 沒有空位 → 逾期
15    }
16    return pen;
17 }
18 // 方法二：期限很大 ( $1e9$ ) 時，依期限排序 + min-heap 存已選工作的懲罰。 $O(N \log
  N)$ 
19 ll min_penalty_heap(vector<pair<int, int>> jobs) {
20     sort(jobs.begin(), jobs.end());
21     priority_queue<int, vector<int>, greater<int>> pq;
22     ll total = 0, kept = 0;
23     for (auto [d, p] : jobs) {
24         total += p; pq.push(p); kept += p;
25         if ((int)pq.size() > d) kept -= pq.top(), pq.pop(); // 排太多，丟掉懲
      罰最小的
26     }
27     return total - kept;
28 }
29 // 耗時不是 1 (工作 {耗時 t, 期限 d})，最多完成幾個：依期限排序，累計時間超過 d
  就丟掉耗時最長的 (max-heap)
```

## 10.8 多機排程 (LPT 近似)

**用法:** 耗時大的先做，交給最早空出來的機器。只是近似解。

**時間複雜度:**  $O(N \log N)$

```
1 // m 台機器、n 個工作，最小化最晚完工時間 (NP-hard)。LPT 近似：耗時大的先做，交
  給最早空出來的機器
2 ll lpt_schedule(vector<ll> jobs, int m) {
3     sort(jobs.rbegin(), jobs.rend());
4     priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
      int>>> pq; // {完工時間, 機器編號}
5     for (int i = 0; i < m; i++) pq.push({0, i});
6     for (ll t : jobs) {
7         auto [done, id] = pq.top(); pq.pop();
8         pq.push({done + t, id});
9     }
10    ll ans = 0;
11    while (!pq.empty()) ans = max(ans, pq.top().first), pq.pop();
12    return ans;
13 }
14 // 注意：這只是近似解！題目要精確最佳解時，可以「對答案二分搜 + 檢查能不能塞進 m
  台」
```

## 10.9 跳躍遊戲 (最少跳幾次)

**用法:**  $a[i]$  為從  $i$  最遠能跳的距離。

**時間複雜度:**  $O(N)$

```
1 // Jump Game II: 站在 i 最遠可以跳到 i + a[i]，從 0 到 n-1 最少跳幾次 (到不了回
  傳 -1)。 $O(N)$ 
2 // 想法：BFS 一層一層擴展，[目前這一跳能到的範圍] 內找下一跳最遠能到哪
3 int min_jumps(const vector<int> &a) {
4     int n = a.size(), jumps = 0, cur_end = 0, farthest = 0;
5     if (n <= 1) return 0;
6     for (int i = 0; i < n - 1; i++) {
7         if (i > farthest) return -1; // 根本走不到 i
8         farthest = max(farthest, i + a[i]);
9         if (i == cur_end) { // 這一跳的範圍用完，必須再跳
10            if (farthest <= i) return -1;
11            jumps++; cur_end = farthest;
12            if (cur_end >= n - 1) break;
13        }
14    }
15    return cur_end >= n - 1 ? jumps : -1;
16 }
```

## 10.10 Josephus 約瑟夫問題

**用法:** 最後存活者。

**時間複雜度:**  $O(N)$ ； $k$  小時  $O(k \log N)$

```
1 // n 個人 (編號 0..n-1) 圍圈，每數到第 k 個就淘汰，回傳最後存活者編號 (0-based)
2 int josephus(int n, int k) { //  $O(n)$ :  $J(1)=0$ ,  $J(i) = (J(i-1) + k) \% i$ 
3     int w = 0;
4     for (int i = 1; i <= n; i++) w = (w + k) \% i;
5     return w;
6 }
7 ll josephus_fast(ll n, ll k) { //  $O(k \log n)$ ，n 很大 ( $1e18$ ) 但 k 小時用
8     if (n == 1) return 0;
9     if (k == 1) return n - 1;
```

```
10     if (k > n) return (josephus_fast(n - 1, k) + k) \% n;
11     ll res = josephus_fast(n - n / k, k) - n \% k;
12     if (res < 0) res += n;
13     else res += res / (k - 1);
14     return res;
15 }
16 /* 變形 (UVA 151 Power Crisis): 一定先淘汰 1 號，再每 k 個淘汰一個，求讓 13 號
  最後存活的最小 k
17 + 1 號先淘汰後剩 n-1 人，從 2 號開始重新編號 0..n-2，13 號變成 11
18 for k = 1...: if (josephus(n - 1, k) == 11) 答案為 k
19 要知道「第 m 個被淘汰的人」：用 ordered_set / BIT 第 k 小模擬， $O(n \log n)$  */
```

## 10.11 Huffman 合併與編碼

**用法:** 每次合併最小的兩堆；陣列存樹產生編碼。

**時間複雜度:**  $O(N \log N)$

```
1 // 合併 n 堆，每次合併代價 = 兩堆重量和，求最小總代價：每次挑最小的兩堆
  (min-heap)
2 ll huffman_cost(const vector<ll> &w) {
3     priority_queue<ll, vector<ll>, greater<ll>> pq(w.begin(), w.end());
4     ll cost = 0;
5     while (pq.size() > 1) {
6         ll a = pq.top(); pq.pop();
7         ll b = pq.top(); pq.pop();
8         cost += a + b; pq.push(a + b);
9     }
10    return cost;
11 }
12 // Huffman 編碼：用陣列存樹 (lc, rc; -1 = 葉子)，葉子 i 對應字元 ch[i]
13 const int MAXN = 600;
14 int lc[MAXN], rc[MAXN], node_cnt;
15 char ch[MAXN];
16 map<char, string> code;
17 void get_code(int u, const string &s) {
18     if (lc[u] == -1) { code[ch[u]] = s.empty() ? "0" : s; return; } // 只有一
      種字元時給 "0"
19     get_code(lc[u], s + "0"); get_code(rc[u], s + "1");
20 }
21 int build_huffman(const string &text) { // 回傳根
22     map<char, int> freq;
23     for (char c : text) freq[c]++;
24     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
      int>>> pq; // {頻率, 節點}
25     node_cnt = 0; code.clear();
26     for (auto [c, f] : freq) ch[node_cnt] = c, lc[node_cnt] = rc[node_cnt] =
      -1, pq.push({f, node_cnt++});
27     while (pq.size() > 1) {
28         auto [f1, a] = pq.top(); pq.pop(); // 最小的放左 (0)
29         auto [f2, b] = pq.top(); pq.pop(); // [改] 題目規定左右/同頻率的順序
      時要改這裡
30         lc[node_cnt] = a; rc[node_cnt] = b;
31         pq.push({f1 + f2, node_cnt++});
32     }
33     int root = pq.top().second;
34     get_code(root, "");
35     return root; // 解碼：從 root 依 0/1 走 lc/rc，走到葉子輸出 ch 並回到 root
36 }
```

## 11 搜尋與技巧

### 11.1 Binary Search 二分搜

**用法:** 找第一個/最後一個可行的值、實數二分。附常見 check 寫法。

**時間複雜度:**  $O(\log(\text{值域}) \times \text{check})$

```
1 /* 二分搜的本質：答案有「單調性」(x 可行 → 比 x 大的都可行，或反過來)
2 題目問「最大值的最小值 / 最小值的最大值 / 最少需要多少才能...」→ 對答案二分搜
  + check()
3 寫 check(x) 時就當作「答案已知是 x」，問題通常會變成簡單的 greedy */
4 bool check(ll x) { return true; } // [改] x 可行嗎？
5
6 // 1. 找第一個 (最小) 可行的 x: check 長得像 F F F T T T
7 ll first_true(ll lo, ll hi) { // 答案在 [lo, hi]；全部不可行會回傳 hi+1
8     hi++;
9     while (lo < hi) {
10         ll mid = lo + (hi - lo) / 2;
11         if (check(mid)) hi = mid; else lo = mid + 1;
12     }
13     return lo;
14 }
15 // 2. 找最後一個 (最大) 可行的 x: check 長得像 T T T F F F
16 ll last_true(ll lo, ll hi) { // 全部不可行會回傳 lo-1
17     lo--;
18     while (lo < hi) {
19         ll mid = lo + (hi - lo + 1) / 2; // +1 才不會死迴圈
20         if (check(mid)) lo = mid; else hi = mid - 1;
21     }
22     return lo;
23 }
24 // 3. 實數：固定跑 100 次 (比 while(r-l>eps) 安全)
25 double real_bs(double lo, double hi) {
26     for (int it = 0; it < 100; it++) {
27         double mid = (lo + hi) / 2;
28         if (check(mid)) hi = mid; else lo = mid;
29     }
30     return lo;
31 }
32 /* 4. 陣列 (要先排序):
33 lower_bound(a, a+n, x) - a: 第一個  $\geq x$  的 index; upper_bound: 第一個  $> x$ 
34 x 出現次數 = upper - lower;  $\leq x$  的個數 = upper_bound(...) - a
35 [常見 check]
36 - 切成  $\leq k$  段，使每段和的最大值最小: check(x) = greedy 切，每段和  $\leq x$ ，算
  段數  $\leq k$ 
```

```

37 - 放 k 個東西，使最近距離最大 (Aggressive cows): check(x) = 從左邊開始，距離
    >= x 就放
38 - 第 k 小的「某種組合」(例：兩兩差、矩陣乘法表)：二分值 x，check = 「<= x 的
    有幾個」>= k
39 - 平均值最大 (分數規劃)：二分 x，每個數減 x 後看能否 >= 0 */
40 // [例] 構造：給前綴和的正負號 (+, -, 0)，求元素皆非 0 時「最大絕對值」的最小值
41 // check(M)：維護前綴和可能範圍 [L, R]，每步 L -= M, R += M 再依符號截斷，L > R
    不可行

```

## 11.2 Ternary Search 三分搜

**用法：**單峰/單谷函數極值。浮點數固定跑 200 次；整數剩 3 個以內暴力。

**時間複雜度：** $O(\log(\text{值域}))$

```

1 // 單峰/單谷函數求極值。以「求最小值」為例；求最大值把 > 改成 <、min 改 max
2 double f(double x) { return (x - 2) * (x - 2); } // [改] 目標函數
3 double ternary_double(double L, double R) { // 回傳極值所在的 x
4     for (int it = 0; it < 200; it++) {
5         double m1 = L + (R - L) / 3, m2 = R - (R - L) / 3;
6         if (f(m1) > f(m2)) L = m1; else R = m2;
7     }
8     return L;
9 }
10 ll fi(ll x) { return (x - 2) * (x - 2); } // 整數版目標函數
11 ll ternary_int(ll L, ll R) { // 回傳最小的函數值
12     while (R - L > 2) {
13         ll m1 = L + (R - L) / 3, m2 = R - (R - L) / 3;
14         if (fi(m1) > fi(m2)) L = m1; else R = m2;
15     }
16     ll best = fi(L);
17     for (ll x = L + 1; x <= R; x++) best = min(best, fi(x)); // 剩 <= 3 個直接暴力
18     return best;
19 }
20 // 注意：函數有「平台」(連續相等的值) 時三分搜可能失敗 → 整數可改二分搜 f(x) 與
    f(x+1) 的大小關係

```

## 11.3 Two Pointer 雙指針

**用法：**同向 (視窗) 與對向 (排序後)。

**時間複雜度：** $O(N)$

```

1 /* 雙指針 (同向)：r 每次往右一格，l 在「條件被破壞」時往右縮。總共 O(N)
2 能用的條件：區間變大條件只會更難滿足 (單調性)，例如「和 <= x 且元素非負」 */
3 const int MAXN = 200005;
4 int n, a[MAXN];
5 // 1. 和 <= x 的最長子陣列 (a 非負)
6 int longest_sum_at_most(ll x) {
7     ll sum = 0; int best = 0;
8     for (int l = 0, r = 0; r < n; r++) {
9         sum += a[r];
10        while (sum > x) sum -= a[l++]; // [改] 縮的條件
11        best = max(best, r - l + 1);
12    }
13    return best;
14 }
15 // 2. 和 >= target 的最短子陣列 (a 非負)，找不到回傳 0
16 int shortest_sum_at_least(ll target) {
17     ll sum = 0; int best = INT_MAX;
18     for (int l = 0, r = 0; r < n; r++) {
19         sum += a[r];
20         while (l <= r && sum >= target) best = min(best, r - l + 1), sum -= a[l++];
21     }
22     return best == INT_MAX ? 0 : best;
23 }
24 // 3. 對向：排序後找兩數和 = x (3 Sum: 固定第一個數，剩下做 2 Sum, O(N^2))
25 bool two_sum(ll x) { // a 已排序
26     for (int l = 0, r = n - 1; l < r; ) {
27         ll s = a[l] + a[r];
28         if (s == x) return true;
29         s < x ? l++ : r--;
30     }
31     return false;
32 }
33 // 有負數時「和 = k 的子陣列」不能用雙指針 → 前綴和 + map
34 // 4. Floyd 判圈 (函數迭代 x → f(x) 找環)：slow 走 1 步、fast 走 2 步，相遇代表
    有環

```

## 11.4 Sliding Window 滑動視窗

**用法：**最長不重複子字串、最多 K 種值的子陣列數。

**時間複雜度：** $O(N)$  (用 map 時  $O(N \log N)$ )

```

1 // 滑動視窗 = 同向雙指針 + 維護視窗內的統計 (計數陣列/map)
2 // 1. 最長「不含重複字元」的子字串
3 int longest_unique(const string &s) {
4     int cnt[256] = {}, best = 0;
5     for (int l = 0, r = 0; r < (int)s.size(); r++) {
6         cnt[(unsigned char)s[r]]++;
7         while (cnt[(unsigned char)s[r]] > 1) cnt[(unsigned char)s[l++]]--;
8         best = max(best, r - l + 1);
9     }
10    return best;
11 }
12 // 2. 「最多 K 種不同值」的子陣列個數 (恰好 K 種 = atMost(K) - atMost(K-1))
13 ll at_most_k_distinct(const vector<int> &a, int k) {
14     map<int, int> cnt; ll res = 0;
15     for (int l = 0, r = 0; r < (int)a.size(); r++) {
16         cnt[a[r]]++;
17         while ((int)cnt.size() > k) { if (--cnt[a[l]] == 0) cnt.erase(a[l]);
            l++; }

```

```

18         res += r - l + 1; // 以 r 結尾的合法子陣列有 r-l+1 個
19     }
20     return res;
21 }
22 // 3. 固定長度 len 的子陣列最小和：先算前 len 個，之後每步 +a[i] -a[i-len]
23 // 視窗內最大/最小值 → 單調隊列 (MonotonicQueue)
24 // 包含 t 所有字元的最短子字串：need[] 計數，滿足時一直縮左邊並更新答案

```

## 11.5 暴力枚舉、回溯、折半枚舉

**用法：**子集合/排列枚舉、回溯剪枝 ( $N$  皇后)、 $N \leq 40$  的折半枚舉。

**時間複雜度：** $O(2^N) / O(2^{N/2}N)$

```

1 // 1. 枚舉所有子集合 (n <= 20)
2 // for (int mask = 0; mask < (1 << n); mask++) for (i) if (mask >> i & 1)
    ... 選了第 i 個
3 // 2. 枚舉所有排列 (n <= 10)：先 sort，do { ... } while
    (next_permutation(a.begin(), a.end()));
4 // 3. 回溯 + 剪枝 (例：N 皇后)
5 int n, cnt;
6 bool col[20], d1[40], d2[40];
7 void queens(int r) {
8     if (r == n) { cnt++; return; } // [改] 找到一組解
9     for (int c = 0; c < n; c++) {
10        if (col[c] || d1[r + c] || d2[r - c + n]) continue; // 剪枝：不合法就
            不往下
11        col[c] = d1[r + c] = d2[r - c + n] = true;
12        queens(r + 1);
13        col[c] = d1[r + c] = d2[r - c + n] = false; // 回溯：還原
14    }
15 }
16 // 剪枝技巧：目前答案已經比 best 差就 return；先試「比較可能成功」的選項；記憶化
    相同狀態
17
18 // 4. 折半枚舉 (Meet in the middle)：n <= 40，子集合和 = target 的個數，
     $O(2^{(n/2)} * n)$ 
19 vector<ll> half_sums(const vector<ll> &v) {
20     vector<ll> s = {};
21     for (ll x : v) { int k = s.size(); for (int i = 0; i < k; i++)
        s.push_back(s[i] + x); }
22     return s;
23 }
24 ll count_subset_sum(const vector<ll> &a, ll target) {
25     int h = a.size() / 2;
26     vector<ll> L = half_sums(vector<ll>(a.begin(), a.begin() + h));
27     vector<ll> R = half_sums(vector<ll>(a.begin() + h, a.end()));
28     sort(R.begin(), R.end());
29     ll res = 0;
30     for (ll x : L) res += upper_bound(R.begin(), R.end(), target - x) -
        lower_bound(R.begin(), R.end(), target - x);
31     return res; // [改] 最接近 target：對每個 x 在 R 中 lower_bound 找最近的
32 }

```

## 12 MISC

### 12.1 安全的 Hash

**用法：**unordered\_map 被特殊測資卡成  $O(N)$  時使用。

**時間複雜度：**均攤  $O(1)$

```

1 // unordered_map 被惡意測資卡成 O(N) 時 (Codeforces 常見)，換成隨機化的 hash
2 uint64_t splitmix64(uint64_t x) {
3     x += 0x9e3779b97f4a7c15;
4     x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5     x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6     return x ^ (x >> 31);
7 }
8 const uint64_t RND = chrono::steady_clock::now().time_since_epoch().count();
9 auto safe_hash = [](uint64_t x) { return splitmix64(x + RND); };
10 unordered_map<ll, int, decltype(safe_hash)> mp(1 << 16, safe_hash);
11 unordered_set<ll, decltype(safe_hash)> st(1 << 16, safe_hash);
12 // pair 當 key：合成一個 ll，例如 (ll)a * 1000000007 + b，或直接用 map (O(log
    N) 通常也夠)
13 // 加速：mp.reserve(1 << 20); mp.max_load_factor(0.25);

```

### 12.2 對拍器 (測資產生器)

**用法：**有一個解法、也有保證正確的暴力解時，用隨機小測資找出反例。產生器：陣列、排列、字串、區間、各種形狀的樹 (隨機/鍵/星/二元/毛毛蟲)、一般圖、連通圖、DAG、完全圖、網格。

```

1 // ===== gen.cpp: 隨機測資產生器。N 與數值都要調小 (N <= 10、值 <= 100)，WA 時
    才看得懂 =====
2 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
3 ll rnd(ll L, ll R) { return uniform_int_distribution<ll>(L, R)(rng); } // [L,
    R] 的隨機整數
4
5 // ----- 1. 陣列與字串 -----
6 void gen_array(int n, ll L, ll R) { for (int i = 0; i < n; i++) cout <<
    rnd(L, R) << " \n"[i == n - 1]; }
7 void gen_permutation(int n) { // 1-n 不重複
8     vector<int> p(n); iota(p.begin(), p.end(), 1); shuffle(p.begin(),
        p.end(), rng);
9     for (int i = 0; i < n; i++) cout << p[i] << " \n"[i == n - 1];
10 }
11 void gen_few_distinct(int n, ll L, ll R, int k) { // 只有 k 種不同的值 (測重複
    值)
12     vector<ll> pool(k); for (ll &x : pool) x = rnd(L, R);
13     for (int i = 0; i < n; i++) cout << pool[rnd(0, k - 1)] << " \n"[i == n -
        1];
14 }
15 void gen_sorted(int n, ll L, ll R, bool rev) { // 已排序 / 完全反序
16     vector<ll> a(n); for (ll &x : a) x = rnd(L, R);

```

```

17     sort(a.begin(), a.end()); if (rev) reverse(a.begin(), a.end());
18     for (int i = 0; i < n; i++) cout << a[i] << " \n"[i == n - 1];
19 }
20 void gen_string(int n, string cs = "abc") { for (int i = 0; i < n; i++) cout
    << cs[rnd(0, cs.size() - 1)]; cout << '\n'; }
21 void gen_intervals(int n, ll mx) { // n 個區間 [l, r]
22     for (int i = 0; i < n; i++) { ll l = rnd(1, mx), r = rnd(1, mx); if (l >
        r) swap(l, r); cout << l << ' ' << r << '\n'; }
23 }
24
25 // ----- 2. 樹 (n 個點、n-1 條邊) -----
26 // 共用：打亂點的編號與邊的順序，避免「編號剛好有順序」讓錯的 code 也 AC
27 void print_tree(int n, vector<pair<int, int>> e, bool weighted = false, ll
    maxv = 10) {
28     vector<int> p(n + 1); iota(p.begin(), p.end(), 0); shuffle(p.begin() + 1,
        p.end(), rng);
29     shuffle(e.begin(), e.end(), rng);
30     for (auto [u, v] : e) {
31         u = p[u]; v = p[v]; if (rnd(0, 1)) swap(u, v);
32         cout << u << ' ' << v; if (weighted) cout << ' ' << rnd(1, maxv);
            cout << '\n';
33     }
34 }
35 void gen_random_tree(int n, bool w = false) { vector<pair<int, int>> e; for
    (int i = 2; i <= n; i++) e.push_back({(int)rnd(1, i - 1), i}); print_tree(n,
    e, w); }
36 void gen_chain(int n, bool w = false) { vector<pair<int, int>> e; for (int i
    = 2; i <= n; i++) e.push_back({i - 1, i}); print_tree(n, e, w); } // 一條線
    (卡遞迴深度)
37 void gen_star(int n, bool w = false) { vector<pair<int, int>> e; for (int i =
    2; i <= n; i++) e.push_back({1, i}); print_tree(n, e, w); } // 一個中心
    (卡 O(度數^2))
38 void gen_binary_tree(int n, bool w = false) { vector<pair<int, int>> e; for
    (int i = 2; i <= n; i++) e.push_back({i / 2, i}); print_tree(n, e, w); } //
    完全二元樹
39 void gen_caterpillar(int n, bool w = false) { // 毛毛蟲：一條主幹，其他點都直
    接掛在主幹上
40     vector<pair<int, int>> e; int len = rnd(1, n);
41     for (int i = 2; i <= len; i++) e.push_back({i - 1, i});
42     for (int i = len + 1; i <= n; i++) e.push_back({(int)rnd(1, len), i});
43     print_tree(n, e, w);
44 }
45
46 // ----- 3. 圖 (n 個點、m 條邊) -----
47 void gen_graph(int n, int m, bool w = false, bool directed = false) { // 無自
    環、無重邊。m <= n(n-1)/2
48     set<pair<int, int>> e;
49     while ((int)e.size() < m) {
50         int u = rnd(1, n), v = rnd(1, n);
51         if (u == v) continue;
52         if (!directed && u > v) swap(u, v);
53         e.insert({u, v});
54     }
55     vector<pair<int, int>> ev(e.begin(), e.end()); shuffle(ev.begin(),
        ev.end(), rng);
56     for (auto [u, v] : ev) { cout << u << ' ' << v; if (w) cout << ' ' <<
        rnd(1, 10); cout << '\n'; }
57 }
58 void gen_connected_graph(int n, int m, bool w = false) { // 保證連通：先產生一
    棵隨機樹，再補 m-(n-1) 條邊
59     set<pair<int, int>> e;
60     for (int i = 2; i <= n; i++) e.insert({(int)rnd(1, i - 1), i});
61     while ((int)e.size() < m) { int u = rnd(1, n), v = rnd(1, n); if (u != v)
        e.insert({min(u, v), max(u, v)}); }
62     vector<int> p(n + 1); iota(p.begin(), p.end(), 0); shuffle(p.begin() + 1,
        p.end(), rng);
63     vector<pair<int, int>> ev(e.begin(), e.end()); shuffle(ev.begin(),
        ev.end(), rng);
64     for (auto [u, v] : ev) { cout << p[u] << ' ' << p[v]; if (w) cout << ' '
        << rnd(1, 10); cout << '\n'; }
65 }
66 void gen_dag(int n, int m, bool w = false) { // 有向無環：只產生「小編號 → 大
    編號」的邊，再打亂編號
67     set<pair<int, int>> e;
68     while ((int)e.size() < m) { int u = rnd(1, n - 1); e.insert({u,
        (int)rnd(u + 1, n)}); }
69     vector<int> p(n + 1); iota(p.begin(), p.end(), 0); shuffle(p.begin() + 1,
        p.end(), rng);
70     vector<pair<int, int>> ev(e.begin(), e.end()); shuffle(ev.begin(),
        ev.end(), rng);
71     for (auto [u, v] : ev) { cout << p[u] << ' ' << p[v]; if (w) cout << ' '
        << rnd(1, 10); cout << '\n'; }
72 }
73 void gen_complete_graph(int n, bool w = false) { gen_graph(n, n * (n - 1) /
    2, w); } // 完全圖 (卡 O(E^2))
74 void gen_grid(int r, int c, int wall_pct = 20) { // 網格地圖：'#' 牆、'.' 空地
75     cout << r << ' ' << c << '\n';
76     for (int i = 0; i < r; i++) { for (int j = 0; j < c; j++) cout << (rnd(1,
        100) <= wall_pct ? '#' : '.'); cout << '\n'; }
77 }
78
79 int main() { // [改] 依題目的輸入格式組合上面的函式
80     int T = 1; // 多筆測資時：T = rnd(1, 3); cout << T << '\n';
81     while (T--> 0) {
82         int n = rnd(2, 8), m = rnd(n - 1, n * (n - 1) / 2);
83         cout << n << ' ' << m << '\n';
84         gen_connected_graph(n, m, true);
85     }
86 }
87
88 /* ===== 使用步驟 =====
89 1. 同一個資料夾放：sol.cpp (你的解)、brute.cpp (暴力但保證對)、gen.cpp、
    check.sh 或 check.bat
90 2. 改 gen.cpp 的 main，產生符合題目格式的「小」測資
91 3. 執行 check，出現 WA 時：in.txt = 讓你錯的測資，ans.txt = 正確答案，out.txt =
    你的答案
92 ===== check.sh (Linux / Git Bash)：第一次先 chmod +x check.sh =====
93 g++ -O2 gen.cpp -o gen && g++ -O2 sol.cpp -o sol && g++ -O2 brute.cpp -o
    brute
94 for ((i = 1; ; i++)); do
95     ./gen > in.txt; ./sol < in.txt > out.txt; ./brute < in.txt > ans.txt
96     if ! diff -w out.txt ans.txt > /dev/null; then echo "WA on test $i"; cat
        in.txt; break; fi
97     echo "Test $i: AC"
98 done
99
100 ===== check.bat (Windows)：先手動編譯成 gen.exe / sol.exe / brute.exe =====
101 :loop
102 gen > in.txt & sol < in.txt > out.txt & brute < in.txt > ans.txt
103 fc /w out.txt ans.txt > nul || (echo WA & type in.txt & exit /b)
104 goto loop
105 */

```